Congelar la web no significa terminar IASI

Javier G. Grandez

2026-09-09

Tabla de contenidos

Journey 6
El recorrido . . . 6

title: “El origen” 7

El origen 8
title: “Por qué contamos el viaje” . . . 11

title: “Cuando el proceso empezó a importar” 16
Lo que quedó . . . 17
Fuentes . . . 17

title: “Guardar antes de interpretar” 18
Una consecuencia inesperada . . . 19
Fuentes . . . 20

title: “Un gráfico que no quería ser un diagrama” 21
Fuentes . . . 23

title: “Primero la especificación, después el código” 24
Fuentes . . . 26

title: “IASI Graphics destapa la metodología” 27
Fuentes . . . 29

title: “Todo se reduce a inputs” 30
Fuentes . . . 32

title: “De metodología escrita a sistema ejecutable” 33
Fuentes . . . 35

title: “Instructions, skills y plataformas” 36
Fuentes . . . 38

title: “El lugar natural del cambio” 39
Lo que quedó . . . 41
Fuentes . . . 41

title: “Cuando OpenSpec dejó de ser necesario” **42**
Fuentes . . . 44

title: “Inputs no es lo mismo que entender” **45**
Fuentes . . . 47

title: “`define` no genera: reconcilia” **48**
Fuentes . . . 51

title: “IASI decide construirse con IASI” **52**
Fuentes . . . 53

title: “Nos perdimos y paramos” **54**
Lo que quedó . . . 56
Fuentes . . . 56

title: “El Journey empieza a escribirse” **57**
Fuentes . . . 58

title: “Publicaciones multidocumento” **59**
El problema . . . 60
La necesidad descubierta . . . 60
Identidad de publicación frente a implementación . . . 61
Compatibilidad directa con la tecnología subyacente . . . 62
Perfiles independientes . . . 63
`all` como selección editorial . . . 64
Build y Publish . . . 65
HTML como punto de entrada . . . 66
Un punto de integración explícito . . . 67
Consecuencia metodológica . . . 67
Necesidad incorporada . . . 68

title: “Los labs empiezan a tomar forma” **69**

title: “Congelar también es ingeniería” **72**
Lo que quedó . . . 73
Fuentes . . . 73

title: “Las reglas se vuelven artefactos” **74**
Lo que quedó . . . 76
Fuentes . . . 76

title: “Lo que IASI lleva consigo” **77**
Lo que quedó . . . 79

Fuentes . . . 79

title: “El código es un output” 80
Lo que quedó . . . 82
Fuentes . . . 82

title: “El prompt deja de ser el contenedor” 83
Lo que quedó . . . 85
Fuentes . . . 85

title: “La infraestructura sale de la máquina de desarrollo” 86
Lo que quedó . . . 87
Fuentes . . . 88

title: “Descargamos modelos. Construimos agentes.” 89
Lo que quedó . . . 90
Fuentes . . . 91

title: “Una presentación nos obliga a aplicar IASI” 92
Lo que quedó . . . 94
Fuentes . . . 95

title: “Un lenguaje visual para IASI” 96
Lo que quedó . . . 98
Fuentes . . . 98

title: “La frontera también se protege” 99
Una simetría en los extremos . . . 102
Lo que apareció . . . 102
Fuentes . . . 103

title: “La documentación también tiene ciclo de vida” 104
Lo que quedó . . . 106
Fuentes . . . 106

title: “Reproducibilidad cuando la construcción es probabilística” 107
Lo que quedó . . . 109
Fuentes . . . 109

title: “El SI aumenta al ingeniero, no al equipo” 110
Lo que quedó . . . 112
Fuentes . . . 112

title: “El presente se convierte en el punto de partida” 113
Lo que quedó . . . 115

Fuentes . . . 115

title: “Los roles son necesarios; los puestos y los recursos son contingentes” 116
Lo que quedó . . . 117
Fuentes . . . 118

title: “El punto de inflexión depende de la escala” 119
Lo que quedó . . . 121
Fuentes . . . 121

title: “Astra se convierte en un caso real para IASI” 122
Lo que quedó . . . 124
Fuentes . . . 124

title: “La estructura del repositorio también es arquitectura” 125
Lo que quedó . . . 128
Fuentes . . . 128

title: “Congelar la web no significa terminar IASI” 129
Congelar sin fingir . . . 131
Lo que quedó . . . 132
Fuentes . . . 132

Journey

IASI no nació de un diseño cerrado ni de una metodología escrita de principio a fin. Fue apareciendo mientras intentábamos resolver problemas concretos, discutir decisiones, probar herramientas, equivocarnos, corregirnos y descubrir que algunas de las piezas que parecían auxiliares eran, en realidad, parte de la metodología.

Este Journey conserva ese recorrido.

No pretende sustituir a los libros, a las `definitions/`, a los ADR/EDR ni al código. Cada uno de esos artefactos responde a otra pregunta.

- Los **libros** cuentan el conocimiento ya consolidado.
- Las **definitions** expresan lo que IASI entiende y mantiene como estado canónico.
- Las **decisiones** fijan lo que se decidió.
- El **código** muestra lo que se construyó.
- El **Journey** conserva cómo llegamos hasta allí.

Por eso aquí aparecerán ideas que después fueron descartadas, nombres que cambiaron, arquitecturas provisionales, preguntas sin respuesta y momentos en los que hubo que parar.

No corregiremos retrospectivamente una entrada para hacer que el pasado parezca más inteligente de lo que fue. Si algo cambia, la historia continuará en otra entrada.

El recorrido

Los capítulos aparecen en el orden en que el proyecto fue cambiando.

No están agrupados en partes ni reorganizados retrospectivamente por temas. La secuencia es deliberada: cada capítulo hereda las preguntas del anterior y deja abiertas las que empujarán al siguiente.

title: “El origen”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

El origen

IASI no nace con la intención de crear una metodología. Tampoco empieza con un repositorio, un conjunto de herramientas o una arquitectura concreta. Durante bastante tiempo ni siquiera existe como proyecto. Lo que hay es una sensación cada vez más difícil de ignorar: estamos empezando a utilizar la inteligencia artificial de una forma que no encaja demasiado bien con la manera en que habitualmente se habla de ella.

Por todas partes se habla de IA. Aparecen nuevos modelos, asistentes integrados en los entornos de desarrollo, generadores de código y una cantidad creciente de cursos, artículos y demostraciones. Sin embargo, buena parte de la conversación sigue girando alrededor del prompt. Cómo escribirlo mejor, cómo conseguir una respuesta más precisa, qué palabras utilizar o cómo construir una biblioteca de prompts reutilizables.

Es útil, por supuesto, pero empieza a resultar insuficiente.

Un prompt permite pedir algo a un modelo. El problema es que trabajar no consiste únicamente en pedir cosas. Un ingeniero trabaja dentro de un contexto, utiliza herramientas, consulta información, toma decisiones, sigue determinadas reglas y produce artefactos que deben mantenerse coherentes con todo lo anterior. Cada nueva conversación con un modelo obliga, de una forma u otra, a reconstruir parte de ese mundo.

Mientras tanto empiezan a aparecer mecanismos que apuntan en otra dirección. Los agentes pueden realizar tareas que requieren varios pasos. Los MCP permiten conectar los modelos con información, herramientas y sistemas externos. Las skills permiten encapsular formas de realizar determinados trabajos. Las instructions pueden establecer cómo debe comportarse un agente más allá de una petición concreta. Los entornos de desarrollo empiezan a integrar todas esas capacidades alrededor del código y del propio workspace.

La diferencia es importante. Ya no estamos únicamente ante un sistema al que formulamos una pregunta y del que esperamos una respuesta. Empezamos a tener sistemas capaces de participar en un proceso de trabajo.

Y eso plantea un problema bastante más interesante que aprender a escribir buenos prompts.

Hay que aprender a trabajar con ellos.

En nuestro caso, el terreno natural es la ingeniería de software. Es donde podemos probar las ideas de verdad y donde resultan más visibles tanto las posibilidades como los problemas. Un Sistema Inteligente puede generar código, analizar un repositorio, proponer una arquitectura, escribir tests o investigar una tecnología en una fracción del tiempo que requiere tradicionalmente

ese trabajo. Pero precisamente por eso empiezan a cambiar algunas de las condiciones sobre las que se construye nuestra manera habitual de hacer ingeniería.

Durante décadas, implementar es caro. Explorar varias soluciones tiene un coste. Rehacer código tiene un coste. Investigar alternativas tiene un coste. Incluso cuando sabemos que una solución puede diseñarse mejor, muchas veces hay que decidir si merece la pena asumir el esfuerzo y el riesgo de tocar lo que ya funciona.

Con Sistemas Inteligentes parte de ese coste empieza a reducirse de forma muy significativa.

No desaparece la complejidad del problema, pero sí puede disminuir mucho el esfuerzo necesario para materializar, comparar o modificar soluciones. Y si cambia ese coste, también empieza a cambiar el lugar donde se concentra el valor del trabajo de ingeniería.

Cada vez resulta menos interesante preguntarse cuánto código puede producir un modelo y más interesante preguntarse qué código debe producir, por qué, con qué información, bajo qué restricciones y cómo comprobamos que lo producido sigue representando correctamente la intención original.

La implementación sigue importando, pero deja de ocupar necesariamente el centro de todo.

El pensamiento adquiere más peso.

Eso nos lleva también a mirar con cierta desconfianza la idea de utilizar los Sistemas Inteligentes simplemente como una nueva herramienta dentro de los procesos existentes. Añadir un asistente de código a una metodología diseñada para un mundo en el que escribir código es una de las actividades más costosas puede mejorar la productividad, pero no responde a la pregunta que empieza a interesarnos.

La cuestión es si el propio proceso de ingeniería debe cambiar.

No sabemos todavía cuánto ni en qué dirección. De hecho, buena parte de IASI empieza a surgir precisamente al intentar responder a esa pregunta mediante proyectos reales. Pero cada vez parece más claro que no basta con colocar IA encima de la forma anterior de trabajar.

También aparece otro problema. La enorme capacidad de estas herramientas puede producir exactamente el efecto contrario al deseado. Es muy fácil generar más código, más documentos, más alternativas y más actividad. La productividad puede confundirse con producción. Si no existe una forma de mantener contexto, intención y control, la velocidad de los agentes puede multiplicar el ruido con la misma facilidad con la que multiplica el trabajo útil.

Por eso empieza a interesarnos menos enseñar a utilizar una determinada herramienta y más encontrar una forma de enseñar a utilizar Sistemas Inteligentes.

La distinción importa. Las herramientas cambian. Los modelos cambian. Los productos que hoy parecen centrales pueden desaparecer o ser sustituidos rápidamente. Una metodología construida alrededor de los detalles de Copilot, Codex, Claude o cualquier otro producto tendría una vida bastante corta.

Lo que buscamos tiene que estar un nivel por encima.

Queremos entender cómo proporcionar contexto de forma adecuada, cómo incorporar conocimiento al proceso sin tener que repetirlo constantemente, cómo separar una petición concreta de las reglas generales de trabajo, cómo utilizar herramientas externas desde un agente, cómo conservar decisiones, cómo permitir que varios Sistemas Inteligentes participen sin convertir cada uno de ellos en una isla y cómo mantener al humano en el lugar donde sigue siendo imprescindible: comprender, cuestionar, decidir y validar.

Los MCP, las skills, las instructions, los agentes o las herramientas no son la metodología. Son piezas que empiezan a hacer posible otra manera de trabajar. Tenemos que entender qué papel desempeña cada una y, sobre todo, evitar que la metodología dependa de la forma concreta que esas piezas adoptan en una plataforma determinada.

Al mismo tiempo seguimos construyendo cosas.

Y probablemente es eso lo que termina empujándonos hacia IASI. Las preguntas no aparecen mientras intentamos escribir una teoría sobre inteligencia artificial. Aparecen mientras intentamos trabajar con ella. Cada proyecto obliga a resolver un problema que inicialmente parece local y que después resulta tener consecuencias mucho más amplias.

Cómo conservar el conocimiento que recibimos. Cómo distinguir lo que viene de fuera de lo que nosotros añadimos. Cómo saber si hay información suficiente para que un agente continúe. Cómo evitar que rellene los huecos inventando decisiones. Cómo expresar una intención sin obligar a quien la implementa a seguir una solución prescrita de antemano. Cómo conseguir que una regla metodológica funcione igual aunque cambiemos de agente. Cómo conservar el proceso que lleva hasta una decisión y no únicamente la decisión final.

Todavía no tenemos respuestas para muchas de esas preguntas y, sobre todo, todavía no sabemos que terminarán formando parte de una misma cosa.

IASI empieza a aparecer poco a poco alrededor de ellas.

El nombre llega después. También los repositorios, los libros, los inputs, las definitions, las skills y todo lo que empieza a crecer a su alrededor. Al principio solo está la intuición de que los Sistemas Inteligentes pueden cambiar mucho más que la velocidad con la que escribimos software, y que aprender a utilizarlos de verdad exige algo más que aprender a conversar con ellos.

Hay que descubrir cómo integrarlos en la ingeniería sin perder precisamente aquello que hace que siga siendo ingeniería.

Ese es el problema en el que nos estamos metiendo.

Y a partir de ahí empieza el viaje.

---

title: “Por qué contamos el viaje”

A medida que IASI empieza a tomar forma aparece un problema que al principio no parece especialmente importante: estamos produciendo resultados, pero estamos perdiendo buena parte de lo que ocurre para llegar hasta ellos.

Una decisión termina en un documento. Una arquitectura termina reflejada en el código. Una idea suficientemente madura puede acabar en uno de los libros. Git conserva perfectamente qué fichero cambia y cuándo cambia. Desde ese punto de vista parece que tenemos todo lo necesario para reconstruir el proyecto.

Pero no es verdad.

Entre el problema inicial y el resultado final ocurre casi todo lo que realmente nos interesa.

Una idea aparece durante una conversación y parece razonable. La discutimos. Encontramos una excepción. Probamos otra posibilidad. Esa alternativa resuelve una cosa y rompe otra. Volvemos atrás. Una pregunta aparentemente secundaria cambia la forma en que entendemos el problema. A veces empezamos intentando resolver una cuestión puramente técnica y terminamos descubriendo algo que afecta a la metodología completa.

Cuando finalmente llegamos a una solución, todo ese recorrido desaparece con enorme facilidad.

La documentación tradicional tiende a contar las cosas desde el final. Una vez conocida la solución resulta natural explicar el problema, presentar la arquitectura elegida y justificar por qué es adecuada. El resultado suele ser limpio, comprensible y útil.

También es inevitablemente engañoso.

No porque lo que cuenta sea falso, sino porque elimina el proceso que conduce hasta allí. Las dudas desaparecen, las alternativas descartadas dejan de existir y las decisiones parecen mucho más evidentes de lo que son mientras las estamos tomando.

Una metodología puede terminar sufriendo exactamente el mismo problema.

Podemos escribir qué debe hacerse, en qué orden, qué artefactos existen y cuáles son sus responsabilidades. Podemos describir el resultado correcto de aplicar IASI. Pero eso no enseña necesariamente cómo se trabaja cuando las cosas todavía no están claras.

Y eso es precisamente lo que queremos entender.

IASI no se está diseñando primero para aplicarlo después. Se está descubriendo mientras trabajamos. Muchas de sus ideas aparecen porque un proyecto real nos obliga a enfrentarnos a un problema que no habíamos previsto. `iasi-graphics`, por ejemplo, empieza siendo una necesidad bastante concreta y termina poniendo sobre la mesa preguntas sobre especificaciones, inputs, agentes, herramientas y la propia forma en que queremos construir software.

Si contamos únicamente el resultado final, esa relación desaparece.

Podremos decir algún día que IASI utiliza `inputs`, `definitions`, skills o determinadas reglas de trabajo. Lo que no podremos ver es por qué aparecen, qué problema intenta resolver cada una, qué alternativas consideramos antes y qué parte de la idea original cambia durante el proceso.

Eso empieza a parecernos una pérdida importante.

Hay además otra diferencia respecto a la forma tradicional de trabajar. Gran parte de este proceso ocurre conversando con Sistemas Inteligentes.

La conversación no es simplemente una interfaz para pedir un resultado. Se convierte en parte del trabajo intelectual. Nosotros planteamos una idea, el sistema propone posibilidades, las cuestionamos, aparecen relaciones que no habíamos visto, descartamos soluciones y volvemos a formular el problema. En otras ocasiones ocurre al revés: el Sistema Inteligente propone una interpretación que parece razonable y somos nosotros quienes detectamos que parte de una premisa equivocada.

Lo interesante no está únicamente en quién tiene la idea correcta.

Está en cómo cambia la comprensión del problema durante esa interacción.

Eso hace que conservar solamente el documento final resulte todavía más pobre. El documento puede mostrar la decisión, pero no muestra la secuencia de razonamiento que permite comprender por qué esa decisión termina existiendo.

Empieza entonces a aparecer la necesidad de conservar dos cosas distintas.

Por un lado está la conversación original. Debe mantenerse como ocurrió. Puede contener repeticiones, errores, enfados, propuestas que duran cinco minutos y frases que fuera de contexto no parecen especialmente relevantes. No importa. Es la fuente. Si la modificamos después para hacerla más limpia, dejamos de conservar lo que realmente ocurrió.

Por otro lado está la narración.

La narración no pretende reproducir cada conversación. Tampoco pretende resumirlas todas. Intenta identificar aquellos momentos en los que cambia algo: nuestra comprensión de un problema, una decisión de arquitectura, una regla metodológica, la relación entre dos componentes o incluso nuestra propia manera de trabajar con los Sistemas Inteligentes.

Pero al intentar conservar el recorrido aparece otra necesidad que, en realidad, no es nueva. Muchas metodologías y prácticas de ingeniería la resuelven de distintas maneras y con distintos nombres: **registrar las decisiones que se van tomando**.

Porque conservar cómo llegamos a una decisión no resuelve por sí solo un problema muy cotidiano.

Semanas después alguien puede volver a plantear exactamente la misma cuestión.

¿Por qué hacemos esto así? ¿Por qué usamos esta tecnología? ¿Por qué descartamos aquella alternativa? ¿Por qué esta responsabilidad está aquí y no allí?

Si la respuesta está únicamente repartida entre conversaciones, documentos y recuerdos de quienes participaron, el equipo vuelve a empezar la discusión. A veces incluso llega exactamente a la misma conclusión después de invertir de nuevo varias horas. Otras veces toma una decisión distinta simplemente porque ya nadie recuerda las razones que justificaron la anterior.

Con Sistemas Inteligentes este problema puede incluso amplificarse. Un agente que entra hoy en un proyecto no ha participado necesariamente en las conversaciones de ayer. Si no existe un lugar donde pueda comprobar qué decisiones están vigentes, puede volver a cuestionar asuntos ya resueltos, proponer alternativas descartadas o reconstruir desde cero un razonamiento que el proyecto ya ha realizado.

No queremos impedir que una decisión pueda cuestionarse.

Todo lo contrario.

Una decisión debe poder revisarse cuando aparece nueva información, cambian las condiciones o descubrimos que las premisas originales ya no son válidas. Lo que no tiene sentido es **reinventar continuamente la rueda porque hemos perdido la memoria de que esa discusión ya ocurrió**.

Ahí empieza a hacerse evidente que el registro de decisiones no es simplemente una buena práctica documental.

Es parte del sistema de conocimiento de una metodología.

Necesitamos poder saber qué se decide, cuándo se decide, por qué se decide, qué alternativas se consideran y, cuando sea necesario, qué decisión posterior sustituye a la anterior.

El Journey y el registro de decisiones se parecen porque ambos nacen del mismo proceso, pero cumplen funciones diferentes.

El Journey puede contar una discusión completa, los rodeos, las dudas y el momento en el que cambia nuestra comprensión.

El registro de decisiones necesita ser mucho más directo.

Dice qué queda decidido.

Uno conserva el viaje.

El otro establece el punto al que hemos llegado.

Esta separación empieza a parecernos importante porque permite mantener dos propiedades que, mezcladas en un único artefacto, resultarían difíciles de conseguir.

El Journey puede permitirse ser narrativo, incompleto, provisional e incluso estar equivocado desde la perspectiva del futuro.

Una decisión registrada debe ser inequívoca respecto al estado que representa. Si después cambia, no necesitamos fingir que nunca existió. Registramos el cambio y mantenemos la trazabilidad entre ambas.

De ahí empieza a surgir una conclusión más amplia para IASI: **toda metodología necesita algún mecanismo explícito para registrar sus decisiones**.

Puede llamarse ADR, registro de decisiones, log, record o adoptar cualquier otra forma. El nombre y el formato son secundarios. Lo importante es que exista como artefacto reconocible de la metodología y que no dependa únicamente de la memoria de las personas, de una conversación perdida o de interpretar retrospectivamente el código.

Es una de esas cosas que parecen administrativas hasta que se piensa en su función real.

Una decisión registrada reduce trabajo futuro.

Evita discusiones repetidas. Permite incorporar a nuevas personas y nuevos agentes. Conserva el contexto que el código no puede expresar. Hace posible revisar una decisión sin tener que reconstruir primero por qué existe. Y, sobre todo, permite distinguir entre cuestionar conscientemente una decisión y simplemente ignorar que ya fue tomada.

El propio desarrollo de IASI nos lleva así a identificar dos artefactos diferentes que nacen casi al mismo tiempo.

Uno existe para que no perdamos **cómo pensamos**.

El otro existe para que no perdamos **qué decidimos**.

Ahí empieza a tener sentido el Journey.

No como un diario en el que anotamos lo que hacemos cada día. Tampoco como una segunda versión de los libros. Y mucho menos como una colección de actas de reuniones.

El Journey cuenta transformaciones.

Un capítulo puede empezar con una necesidad aparentemente pequeña y terminar con una idea que modifica una parte importante de IASI. Otro puede contar una solución que creemos correcta durante un tiempo y que después abandonamos. Otro puede no terminar con ninguna respuesta, simplemente con una pregunta que tarda varias etapas del viaje en resolverse.

Eso implica aceptar algo que no suele resultar cómodo en la documentación técnica: el Journey puede contener cosas que más adelante dejan de ser ciertas.

Si en un momento creemos que necesitamos una determinada herramienta, eso forma parte del viaje. Si después descubrimos que no la necesitamos, no volvemos al capítulo anterior para corregirlo y hacer que parezca que siempre lo supimos. Continuamos la historia desde el nuevo punto.

De esa forma cada capítulo conserva no solo información sobre IASI, sino también el estado de nuestra comprensión en ese momento.

La fecha importa, pero no organiza el relato. No queremos escribir una secuencia de “hoy hicimos esto” y “mañana aquello”. La unidad narrativa es el episodio, el momento en el que una idea se mueve.

A veces ese episodio ocurre en una tarde. Otras veces atraviesa varias conversaciones o varios proyectos.

Lo que importa es que entre el principio y el final del capítulo algo haya cambiado.

Esta forma de conservar el viaje tiene además una utilidad que empieza a resultar evidente mientras IASI crece.

Las decisiones registradas nos permiten saber qué hemos decidido. Los artefactos nos muestran qué hemos construido. Los libros pueden explicar aquello que ya consideramos conocimiento consolidado.

El Journey conserva otra cosa.

Conserva el camino que conecta todo lo anterior.

Y esa diferencia es especialmente importante si lo que intentamos construir es una metodología.

No queremos presentar IASI como una colección de reglas que aparecen terminadas de algún sitio. Queremos poder seguir el camino que lleva hasta ellas, ver qué problemas intentan resolver, dónde nos equivocamos y qué ocurre cuando las ponemos delante de un proyecto real.

Porque la forma en que estamos trabajando también forma parte de lo que estamos intentando aprender.

Por eso contamos el viaje mientras todavía estamos dentro de él.

title: “Cuando el proceso empezó a importar”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Al principio, casi todo lo que producíamos tenía una forma conocida: repositorios, libros, documentación, código, decisiones. Eran resultados. Cosas que podían quedar limpias, ordenadas y explicadas después.

Pero el trabajo real no se parecía demasiado a esos resultados.

Las ideas aparecían en una conversación. Una propuesta parecía buena durante diez minutos y luego dejaba de serlo. Un nombre llevaba a otro. Una solución técnica abría una pregunta metodológica que no estaba en el problema original. A veces una frase informal contenía más información sobre la decisión que el documento final que escribíamos después.

Empezó a aparecer una incomodidad: si conservábamos únicamente el resultado, estábamos perdiendo la parte más interesante.

No era un problema de auditoría. Git ya podía decir qué fichero había cambiado. Tampoco era un problema de documentación. Los libros podían explicar perfectamente el estado final.

El problema era otro:

> ¿Cómo se había llegado hasta allí?

Esa pregunta resultó ser mucho más importante de lo que parecía.

Una metodología suele escribirse desde el final. Cuando todo ha encajado, se eliminan las dudas, se suavizan los giros y se presenta una secuencia limpia. El lector recibe una ruta recta aunque el descubrimiento real haya sido un laberinto.

IASI estaba naciendo precisamente en ese laberinto.

Por eso empezó a tomar forma una separación que después sería fundamental:

```
Journey      → cómo pensamos, discutimos y descubrimos
Decisiones   → qué decidimos
Books        → qué conocimiento consolidamos
Artefactos   → qué construimos
```

No eran cuatro formas de documentar lo mismo. Eran cuatro clases distintas de evidencia.

El Journey tenía además una propiedad que los demás no necesitaban: debía poder estar equivocado.

Si en un momento pensábamos que una arquitectura era correcta, la entrada debía conservarlo. Si dos días después descubríamos que no funcionaba, no se reescribía el pasado. Se añadía el siguiente episodio.

Eso cambia completamente la naturaleza del documento.

El Journey no es una especificación mutable que siempre representa la verdad actual. Es una secuencia de estados de comprensión. Cada entrada dice, en esencia:

> Con lo que sabíamos entonces, esto fue lo que vimos.

Y ahí apareció otra idea que terminaría siendo central: las conversaciones debían conservarse como fuente inmutable. La narración podía interpretar. El chat bruto no.

Así, por primera vez, el proceso dejó de ser el residuo de construir IASI y empezó a convertirse en uno de sus productos.

No porque cada discusión merezca ser publicada. Muchas no lo merecen. Sino porque, cuando una idea cambia la arquitectura, la metodología o nuestra manera de trabajar, el camino que la produjo contiene conocimiento que el resultado final ya no puede mostrar.

Lo que quedó

El Journey no documentaría el estado de IASI.

Documentaría **su evolución**.

Y no intentaría demostrar que habíamos tenido razón desde el principio.

Precisamente lo contrario.

Fuentes

- Memoria de trabajo de IASI, 2026-08-13.
- `2026-08-13-iasi-graphics-chat.md`.
- Conversaciones sobre la separación entre Journey, decisiones, libros y artefactos.

title: “Guardar antes de interpretar”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

La primera necesidad práctica del Journey no fue escribir.

Fue **guardar**.

Mientras trabajábamos con ChatGPT, Codex y otras herramientas, las conversaciones empezaban a contener demasiadas decisiones como para confiar en que siempre podríamos reconstruirlas después. Había chats de móvil, chats de ordenador, tareas de Codex y conversaciones que atravesaban varios problemas a la vez.

La reacción inicial fue muy simple:

> Quiero bajarme todas las conversaciones.

Parecía una tarea de archivo. Terminó definiendo una arquitectura de conocimiento.

Porque enseguida apareció una distinción:

```
raw
treated
```

El material `raw` debía conservarse tal como había ocurrido. Sin reescribir frases, sin ordenar retrospectivamente la discusión, sin quitar contradicciones porque ahora resultaran incómodas.

El material `treated`, en cambio, podía evolucionar. Podíamos resumirlo, relacionarlo con otras conversaciones, extraer decisiones, escribir una entrada de Journey o convertir una conclusión en una definition.

La regla emergió casi sola:

> **Raw nunca cambia. Treated puede evolucionar todas las veces que sea necesario.**

Eso resolvía un problema que todavía no habíamos formulado explícitamente: cómo permitir interpretación sin perder evidencia.

Si solo guardábamos los documentos tratados, cualquier narración futura dependería de la última interpretación. Si solo guardábamos los chats, tendríamos un archivo enorme pero poco navegable.

Necesitábamos ambas cosas.

```
conversación original
        ↓
      raw
        ↓
interpretación
        ↓
journey / decisions / definitions / books
```

Esta separación resultó especialmente importante cuando Codex entró de lleno en el trabajo. Sus tareas no eran simplemente otra ventana de ChatGPT. Tenían su propio contexto, sus actualizaciones intermedias y una relación directa con el workspace y los cambios del código.

De pronto el desarrollo de IASI estaba repartido entre personas, agentes y repositorios.

El Journey no podía depender de recordar después qué había dicho quién.

Por eso los chats se convirtieron en una forma de evidencia.

No porque cada línea sea importante. Muchísimas no lo son. Pero la conversación conserva algo que los artefactos finales pierden: **la causalidad**.

Un fichero puede mostrar que añadimos `inputs/`.

El chat puede mostrar por qué, qué alternativas descartamos, qué entendíamos por input en ese momento y qué pregunta provocó el cambio.

Ese es el material que necesita el Journey.

Una consecuencia inesperada

Guardar conversaciones parecía una tarea administrativa.

En realidad estaba definiendo un principio de IASI:

> Antes de transformar conocimiento, conserva la fuente.

Más tarde aparecerían `inputs/`, `definitions/`, reconciliación semántica y trazabilidad. En ese momento todavía no teníamos todos esos nombres.

Pero la intuición ya estaba ahí.

Primero conservar.

Después interpretar.

Fuentes

- `2026-08-13-iasi-graphics-chat.md`.
- `trabajar-en-todo-iasi-org.md`.
- Conversaciones de 2026-08-13 sobre exportación de ChatGPT y Codex.

title: “Un gráfico que no quería ser un diagrama”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

`iasi-graphics` no empezó como un proyecto de lenguaje ni como un experimento metodológico.

Empezó con una molestia visual.

Ya teníamos PlantUML para diagramas técnicos. Funcionaba bien para lo que era. Pero había figuras que pedían otra cosa: composiciones más editoriales, más cercanas a una presentación. Cajas, círculos, relaciones simples, una idea explicada visualmente. No un UML disfrazado.

La primera búsqueda fue instrumental:

> ¿Con qué hacemos esos dibujos?

Aparecieron Inkscape, Figma, PowerPoint, generación de PPTX, JavaScript. Todos podían resolver una parte, pero chocaban con algo que todavía no habíamos convertido en principio.

Podíamos generar una presentación por código, sí.

Pero entonces el usuario tendría que pensar en el mecanismo gráfico.

Y eso era exactamente lo que no queríamos.

La frase que cambió el problema fue aproximadamente esta:

> La trampa la has puesto en el JavaScript.

Habíamos conseguido una imagen declarando algo en apariencia sencillo, pero la complejidad real seguía escondida en un código específico preparado para esa composición. No habíamos creado un sistema. Habíamos movido la dificultad de sitio.

Entonces apareció otra posibilidad:

> ¿Y si escribimos **qué** queremos representar y el motor decide **cómo** dibujarlo?

Eso abrió un mundo distinto.

Ya no hablábamos de automatizar PowerPoint. Hablábamos de un lenguaje declarativo.

```
intención visual
      ↓
DSL
      ↓
motor
      ↓
SVG / PNG
```

El usuario no debía especificar coordenadas, tamaños, CSS ni primitivas gráficas. Debía expresar conceptos: flujo, comparación, ecosistema, elementos, relaciones, énfasis.

La complejidad gráfica pertenecía al motor.

Esto encajaba con una idea que IASI repetiría después en otros contextos: una buena ingeniería puede ser muy compleja por dentro sin trasladar esa complejidad al consumidor.

La discusión sobre YAML duró poco porque el punto no era YAML. El punto era encontrar una sintaxis que pudiera escribirse, versionarse y, sobre todo, insertarse naturalmente en un documento Quarto.

Incluso la forma de invocarlo importaba:

no queríamos un bloque extraño pegado desde fuera, sino algo que pareciera parte natural del ecosistema de bloques ejecutables.

`iasi-graphics` comenzó así a adquirir una identidad propia:

- entrada textual;
- lenguaje declarativo;
- motor independiente;
- salida gráfica;
- integración con Quarto;
- implementación preferentemente en Go;
- nada de obligar al usuario a escribir JavaScript.

Lo curioso es que todavía creíamos que estábamos diseñando un pequeño artefacto gráfico.

En realidad, acabábamos de crear el problema perfecto para descubrir cómo debía funcionar IASI.

Fuentes

- 2026-08-13-iasi-graphics-chat.md.
- chatgpt-20260813215000-iasi-graphics-codex.md.

title: “Primero la especificación, después el código”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Cuando nació `iasi-graphics`, el repositorio estaba prácticamente vacío.

Había una carpeta:

```
inputs/
```

y dentro, documentos que describían lo que queríamos construir.

Eso produjo una escena bastante reveladora.

La conversación empezó a deslizarse hacia comandos de compilación, tests y estructura Go antes de que existiera código. La reacción fue inmediata:

> ¿Qué coño va a construir en un directorio vacío?

La corrección parecía trivial, pero fijó una manera de trabajar.

No había que darle a Codex una estructura inventada para que empezara a picar código. Había que darle los documentos y pedirle que reconstruyera la intención.

Así surgió el primer experimento serio:

```
inputs/
   ↓
Codex lee
   ↓
explica qué cree que hay que construir
   ↓
identifica arquitectura, vertical slice y ambigüedades
```

Con una condición importante:

no implementar nada todavía.

Queríamos comprobar si la especificación era suficiente para que otro agente entendiera el producto sin que nosotros le sopláramos la respuesta en el prompt.

Eso convertía a Codex en algo más interesante que un generador de código. Se convertía en una prueba de nuestra propia claridad.

Si interpretaba mal el producto, el primer sospechoso no debía ser Codex.

Debían ser los inputs.

La pregunta cambió de:

> ¿Puede Codex programar esto?

a:

> ¿Nuestros documentos contienen suficiente información para que alguien distinto de nosotros reconstruya correctamente lo que queremos?

Codex respondió identificando capacidades, arquitectura, un vertical slice y, sobre todo, ambigüedades.

Y ahí ocurrió algo importante.

Las ambigüedades no se corrigieron editando los documentos originales.

Se añadió nueva información.

Los inputs originales eran evidencia de entrada. Las aclaraciones también eran inputs, pero de otra naturaleza.

Todavía no habíamos llegado a `externals/` e `internals/`, pero la necesidad ya estaba ahí.

Esta pequeña prueba introdujo varios principios que después aparecerían formalmente en IASI:

- la implementación no debe comenzar si falta información esencial;
- un agente no debe inventar decisiones para rellenar huecos;
- las ambigüedades detectadas son información útil;
- la fuente original no se modifica para hacer que parezca más completa de lo que era;
- el conocimiento puede crecer por capas.

Y, quizá lo más importante:

> La especificación no es un documento que acompaña al software. Debe ser capaz de **producir** software.

No de forma mágica ni determinista. Pero sí con suficiente precisión para que un agente diferente pueda pasar de intención a implementación sin depender de una conversación privada que nadie más puede reconstruir.

`iasi-graphics` fue la primera vez que intentamos comprobarlo en serio.

Fuentes

- `chatgpt-20260813215000-iasi-graphics-codex.md`.

title: “IASI Graphics destapa la metodología”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Al día siguiente, `iasi-graphics` dejó de ser solo un proyecto gráfico.

La conversación empezó con una idea técnica: montarlo de forma parecida a PlantUML. Enviar una descripción a un servidor y recibir el resultado.

Eso permitía empujar toda la complejidad al servidor:

```
cliente ligero
     ↓
descripción
     ↓
servidor iasi-graphics
     ↓
SVG o PNG
```

El servidor podía usar Go y cualquier otra herramienta que necesitara. Quarto podía integrarse mediante un filtro Lua. Un agente podía utilizarlo mediante MCP. Incluso un motor de generación de imágenes podría incorporarse detrás sin cambiar el contrato del consumidor.

Era una buena arquitectura de producto.

Pero entonces apareció una observación más importante:

> Estamos construyendo en base a especificaciones.

Y casi inmediatamente:

> `iasi-graphics` hay que documentarlo en paralelo como lab o como decidamos llamarlo.

La palabra *lab* era provisional. La idea no.

Había tres cosas ocurriendo al mismo tiempo:

```
1. Estamos construyendo iasi-graphics.
2. Estamos descubriendo una metodología mientras lo hacemos.
3. Necesitamos conservar el viaje que explica ese descubrimiento.
```

`iasi-graphics` se convirtió así en la PoC de algo que todavía no tenía repositorio completo.

No queríamos escribir primero una metodología teórica y luego buscar un ejemplo que la demostrara. Queríamos que el caso real obligara a la metodología a responder preguntas de verdad.

El primer golpe llegó con `inputs`.

Al principio podía parecer que `inputs/` era simplemente la carpeta donde habíamos puesto nuestras especificaciones.

Pero Javier hizo una precisión decisiva:

> Los inputs son los documentos que ya existen, en el formato que sea: caso de negocio, funcional, etc. Son entradas externas.

Eso impedía diseñar IASI suponiendo que la información llegaría ya limpia, estructurada y escrita para la metodología.

Luego apareció una segunda clase de entrada.

Codex había analizado los documentos y había detectado ambigüedades. Nosotros respondíamos con aclaraciones. Esas aclaraciones también alimentaban el proyecto, pero no eran documentos externos.

Así nacieron conceptualmente:

```
inputs/
  externals/
  internals/
```

Y poco después apareció una tercera fuente: información obtenida por ingeniería inversa, análisis o trabajo del propio sistema.

```
inputs/
  externals/
  internals/
  obtained/
```

La metodología empezaba a materializarse porque un proyecto real la estaba obligando a distinguir clases de conocimiento.

`iasi-graphics` había dejado de ser solo el primer consumidor.

Se había convertido en el instrumento con el que estábamos descubriendo IASI.

Fuentes

- `chatgpt-20260814084622-iasi-graphics-metodologia.md`.

title: “Todo se reduce a inputs”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

La palabra `inputs` parecía demasiado sencilla para sostener una parte importante de una metodología.

Precisamente por eso funcionó.

Habíamos empezado distinguiendo documentos externos de aclaraciones internas:

```
inputs/
  externals/
  internals/
```

La regla era clara: los externos no se modificaban desde IASI. Si un documento externo estaba incompleto, no se “arreglaba” para facilitar el trabajo. Se añadía un input interno que aclaraba lo necesario.

Esto permitía conservar la diferencia entre:

> lo que recibimos

y

> lo que añadimos para poder trabajar con ello.

Pero los `internals/` tenían un problema distinto.

Estaban vivos.

Una aclaración podía ser necesaria durante una iteración y dejar de ser válida en la siguiente. Borrarla destruía historia. Mantenerla activa contaminaba el estado actual.

La solución apareció con una palabra:

```
archived/
```

No hacía falta una carpeta `active/`. Lo activo era simplemente lo que estaba en el lugar normal. Lo que dejaba de participar en la interpretación pasaba al histórico.

Después apareció `obtained/`.

Había conocimiento que no venía de fuera ni era una decisión añadida manualmente. Podía surgir de ingeniería inversa, análisis del código existente, inspección de infraestructura o trabajo del propio proceso.

También era entrada para fases posteriores.

Así la estructura quedó:

```
inputs/
   externals/
   internals/
   obtained/
```

con históricos asociados cuando fueran necesarios.

La simplificación produjo una frase reveladora:

> Con lo que todo se reduce a un directorio: `inputs`.

No significaba que todo conocimiento fuera igual.

Significaba que, para IASI, todo lo que alimenta el razonamiento entra por un mismo concepto.

El formato tampoco debía imponerse demasiado pronto.

Un input podía ser Markdown, una especificación funcional, un documento de negocio, información obtenida de un sistema o cualquier artefacto que aportara conocimiento.

La metodología no debía exigir que el mundo exterior se adaptara a ella antes de poder empezar a pensar.

Entonces apareció la primera etapa formal:

> **Valida. ¿Es suficiente?**

La pregunta de validación no era si los documentos estaban perfectos.

Era si existía suficiente conocimiento para avanzar sin inventar decisiones importantes.

Y apareció una regla de gate que después crecería mucho:

```
si validate falla
    ↓
no se avanza
```

Incluso un input interno deliberadamente parcial podía permitir avanzar si decía, por ejemplo:

> por ahora créame un proyecto Quarto.

La suficiencia dependía de la siguiente fase, no de una fantasía de documentación completa.

Ahí IASI empezó a parecerse menos a una colección de buenas prácticas y más a una máquina de trabajo.

Fuentes

- `chatgpt-20260814084622-iasi-graphics-metodologia.md`.

title: “De metodología escrita a sistema ejecutable”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Hubo un momento en que la distinción se volvió inevitable:

el Volumen III podía **explicar** IASI, pero no podía **ser** IASI.

Una metodología descrita en un libro todavía obliga a cada persona o agente a interpretar cómo aplicarla. Si queríamos que distintos agentes trabajaran de manera consistente, había que materializarla.

La idea apareció alrededor de los comandos.

No comandos de Codex.

No prompts particulares de Copilot.

Comandos de IASI.

```
/validate
/status
/work
/archive
/obtain
```

Los nombres todavía podían cambiar. Lo importante era quién era dueño de su significado.

IASI.

Eso llevó a separar el sistema conceptual de sus plataformas de ejecución.

```
IASI
   commands/
   skills/
   mcp/
   core/
   schemas/
   adapters/
       codex/
       copilot/
       ...
```

Codex podía soportar una forma de comando. Copilot otra. Mañana aparecería otra herramienta.

La metodología no debía convertirse en una colección de instrucciones escritas para el producto de moda de ese mes.

La dirección correcta de dependencia era la contraria:

```
plataforma
   ↓
adaptador
   ↓
IASI
```

Esta decisión cambió también la función del repositorio `iasi`.

Hasta entonces los libros explicaban la metodología y varios artefactos la apoyaban. Ahora hacía falta un lugar que contuviera **su materialización operativa**.

El repositorio `iasi` debía convertirse en eso.

No necesariamente un único ejecutable monolítico, ni una arquitectura completamente decidida desde el primer día. Pero sí la fuente canónica de:

- comandos;
- skills;
- instrucciones;
- validaciones;
- estructura de conocimiento;
- integración agentic;
- runtime cuando fuera necesario.

La diferencia empezó a formularse así:

El Volumen III explica y define la metodología.
`iasi` la implementa.

Y `iasi-graphics` seguía desempeñando un papel crucial.

Era el proyecto que obligaba a comprobar si esa materialización servía para algo.

La metodología no iba a escribirse en una torre de marfil y después aplicarse.

Iba a ser descubierta bajo carga.

Fuentes

- `chatgpt-20260814084622-iasi-graphics-metodologia.md`.
- `chatgpt-20260814104100-iasi-comandos-skills-mcp.md`.

title: “Instructions, skills y plataformas”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

La palabra *agentic* empezó a quedarse pequeña casi en cuanto intentamos usarla.

Sabíamos que había algo que describía cómo debía trabajar un agente. También había operaciones que el humano podía solicitar, procedimientos especializados y herramientas externas.

Meterlo todo bajo una sola carpeta habría funcionado durante unos días.

Después habría empezado la confusión.

La discusión con Codex y Copilot ayudó a separar las piezas:

```
Instructions → cómo debe comportarse el agente
Skills       → cómo se hace bien una clase de tarea
Commands     → qué tarea se solicita ahora
Tools        → qué capacidad ejecutable puede utilizar
MCP          → cómo accede a capacidades externas
Agent        → quién razona y ejecuta
```

La distinción más importante no era terminológica.

Era de propiedad.

Cuando preguntamos si Codex soportaba skills, apareció una tentación evidente: diseñar las skills directamente para Codex.

Pero eso habría invertido la arquitectura.

La conclusión fue:

> Una skill IASI no es una skill de Codex.

Codex puede tener un adaptador capaz de representar una skill IASI en su formato. Copilot puede necesitar otro. Otra plataforma futura podrá utilizar una tercera forma.

```
IASI skill
      adapter Codex
      adapter Copilot
      ...
```

La skill es producto.

La plataforma es entorno de ejecución.

Poco después afinamos otra categoría.

No era `agentics/` lo que buscábamos para las reglas generales.

Era:

```
instructions/
```

Ahí debían vivir cosas como:

- no inventar información ausente;
- validar antes de modificar;
- respetar el alcance;
- tratar explícitamente la incertidumbre;
- normas de escritura cuando se generan documentos;
- límites de decisión del agente.

Eso permitió evitar un error habitual: esconder reglas globales dentro de cada prompt.

Si una norma forma parte de IASI, debe existir una vez en IASI.

Después vendría el problema de instalación.

Una metodología operativa necesita llegar al proyecto donde se utiliza.

La comparación con OpenSpec ayudó:

```
iasi
   ↓ install
proyecto / workspace
```

Pero también aquí apareció una diferencia útil.

En un proyecto concreto podía instalarse únicamente lo necesario. En un workspace genérico, donde todavía no sabemos qué trabajo aparecerá, tenía sentido instalar el conjunto completo.

Todavía estábamos resolviendo mecánica, CLI y rutas.

Pero la arquitectura conceptual ya había quedado bastante limpia:

> IASI define el comportamiento y las capacidades.
> Las plataformas se adaptan a IASI, no IASI a las plataformas.

Fuentes

- `chatgpt-20260815133533-iasi-agentics-skills-codex.md`.
- `chatgpt-20260815135800-iasi-agentics-instructions-cli.md`.

title: “El lugar natural del cambio”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

La discusión empezó con una nueva necesidad.

Una de esas frases inocentes que, en software, pueden terminar creando una capa, un parámetro, una excepción y tres años de mantenimiento.

La reacción habitual es preguntar:

> ¿Dónde metemos esto?

Pero después de años diseñando y revisando sistemas, Javier planteó otra vara de medir:

> **Cuando aparece una nueva necesidad, debe ir a su sitio natural, que además es intuitivo. Si no es así, tu sistema está mal.**

La frase tenía más profundidad de la que aparentaba.

Un sistema no es extensible porque tenga muchos puntos de extensión. Es extensible cuando las nuevas variantes encuentran un lugar que parece haber estado esperándolas, aunque nadie conociera esa variante cuando se diseñó la arquitectura.

El ejemplo bancario lo hizo concreto.

En un sistema de préstamos y créditos existían numerosos productos numéricos. Para incorporar una nueva forma no era necesario modificar el núcleo y añadir condiciones.

Se creaba un módulo con una nomenclatura asociada al producto.

El sistema lo descubría.

Nada más.

Eso llevó a una formulación mucho más precisa:

> **No diseñaste los productos futuros. Diseñaste el lugar natural donde los productos futuros podrían existir.**

No se trata, por tanto, de predecir todas las necesidades.

Se trata de identificar correctamente los ejes de variación.

La conversación había empezado por otra observación: muchas arquitecturas envejecen acumulando excepciones.

Cada una puede haber sido razonable.

El problema está en la serie:

```
nueva necesidad
   ↓
pequeña excepción
   ↓
otra necesidad
   ↓
otra excepción
   ↓
otra capa
```

Al final, parte de lo que llamamos complejidad *legacy* no representa la complejidad del dominio.

Representa el coste histórico de no haber podido rehacer soluciones anteriores.

Y aquí entraron los Sistemas Inteligentes.

Históricamente, un arquitecto podía detectar que una abstracción ya no era correcta y aun así decidir no tocarla. Comprender todas las dependencias, decisiones antiguas, efectos laterales y riesgos podía costar demasiado.

Si un Sistema Inteligente reduce radicalmente el coste de comprender un sistema, cambia la economía de la refactorización.

La pregunta deja de ser:

> ¿Cómo añadimos la sexta excepción sin romper nada?

y puede convertirse en:

> ¿Sigue siendo correcta esta abstracción? ¿Dónde estaría ahora el lugar natural de esta necesidad?

Esto encajó con otra tesis central de IASI:

> Si la implementación cada vez cuesta menos, el pensamiento cada vez vale más.

La IA no cambia la ingeniería únicamente porque escriba código más deprisa.

Puede cambiarla porque hace económicamente posible **volver a pensar** sistemas que antes solo podíamos seguir parcheando.

Lo que quedó

Una buena arquitectura no adivina el futuro.

Hace espacio para el cambio.

Y cuando lo nuevo deja de encontrar su sitio natural, la primera reacción no debería ser inventarle uno.

Debería ser cuestionar el modelo.

Fuentes

- `chatgpt-20260816001600-tenemos-una-nueva-necesidad-refactorizacion.md`.
- `chatgpt-20260816003400-arquitectura-refactorizacion-lugar-natural-del-cambio.md`.
- Memoria de IASI sobre el principio «Todo se debe cuestionar».

title: “Cuando OpenSpec dejó de ser necesario”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

OpenSpec llevaba un tiempo formando parte del paisaje mental de IASI.

No necesariamente como dependencia definitiva, pero sí como una pieza razonable entre los documentos de entrada y la implementación.

Hasta que apareció una pregunta muy pequeña:

> Tenemos nuestros inputs. ¿Para qué necesitamos OpenSpec, al menos por ahora?

La pregunta hizo algo muy IASI: obligó a justificar una pieza que ya habíamos dado por incorporada.

El dibujo inicial podía expresarse así:

```
conocimiento
   ↓
inputs
   ↓
OpenSpec
   ↓
agente
   ↓
implementación
```

Pero IASI había evolucionado.

Los `inputs/` ya eran legibles por humanos y agentes, versionables, independientes de plataforma y suficientemente flexibles para recibir conocimiento sin obligar a convertirlo primero a otro formato.

Así que la pregunta correcta pasó a ser:

> ¿Qué problema real resuelve hoy OpenSpec que IASI no pueda resolver sin él?

La respuesta fue incómodamente sencilla:

ninguno imprescindible.

Eso no convirtió OpenSpec en una mala herramienta.

Simplemente convirtió su incorporación actual en arquitectura anticipada.

```
inputs → OpenSpec → agente
```

añadía una transformación que todavía no necesitábamos.

Podía volver a tener sentido más adelante: cientos de documentos, trazabilidad formal compleja, cobertura, gestión estricta de cambios, interoperabilidad con un ecosistema que justificara el coste.

Pero IASI no debía construir esa infraestructura por si acaso.

Esta decisión fue importante por una razón que trascendía OpenSpec.

Estábamos empezando a aplicar a IASI su propio principio:

> **Todo se debe cuestionar.**

No preguntar:

> ¿Cómo integramos OpenSpec?

sino:

> ¿Qué necesitamos para resolver el problema?

Y solo después comprobar si una herramienta existente satisface esa necesidad.

La retirada provisional de OpenSpec simplificó el flujo y dejó al descubierto el siguiente problema real.

Si los inputs alimentaban directamente a IASI, necesitábamos saber si eran suficientes y coherentes para avanzar.

Así apareció con claridad `/validate`.

La eliminación de una pieza no dejó un hueco.

Reveló la pieza que realmente necesitábamos.

Fuentes

- chatgpt-20260817130542-iasi-workflow-inputs-definitions.md.

title: “Inputs no es lo mismo que entender”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Durante varios días `inputs/` había demostrado ser una abstracción sorprendentemente resistente.

Todo lo que alimentaba al proceso podía entrar allí.

Pero precisamente esa flexibilidad creó el siguiente problema.

Un input humano no tiene por qué venir estructurado como necesita la ingeniería.

Puede mezclar intención, requisito, restricción, decisión, preguntas abiertas y contexto en una sola página. Dos documentos pueden hablar de la misma cosa con nombres distintos. Una conversación puede contener tres conceptos que deberían evolucionar por separado.

IASI necesitaba una representación intermedia.

No un formato impuesto a quien entrega los documentos.

Una representación de **lo que IASI ha entendido**.

La idea ya había aparecido días antes como un posible adaptador entre inputs y OpenSpec. Al desaparecer la necesidad inmediata de OpenSpec, la representación intermedia dejó de ser un adaptador para otra herramienta y adquirió sentido propio.

Probamos mentalmente varios nombres.

`specs/` era tentador, pero acercaba demasiado el concepto a OpenSpec y a la idea de especificación formal.

Terminamos, por ahora, en:

```
definitions/
```

La semántica quedó extraordinariamente sencilla:

```
inputs/       → lo que entra en IASI
definitions/  → lo que IASI ha entendido
```

Esa distinción cambió varias cosas.

Primero, desapareció la idea de una transformación uno a uno.

```
inputs/a.md
inputs/b.md   → definitions/authentication.md
inputs/c.md
```

Varias fuentes pueden describir un mismo concepto.

Y al revés:

```
inputs/request.md
        → definitions/intent.md
        → definitions/scope.md
```

Un solo input puede contener varias unidades semánticas independientes.

Segundo, las definitions debían mantener trazabilidad.

No bastaba con producir una versión bonita y olvidar de dónde había salido.

Tercero, IASI no podía utilizar esa fase para inventar lo que faltaba.

Podía reformular, ordenar, consolidar y hacer explícito aquello que estuviera soportado por las fuentes.

Pero una contradicción seguía siendo una contradicción.

Y una ausencia importante seguía siendo una ausencia.

En vez de ocultarlas, `definitions/` debía poder representarlas.

Ahí empezaron a aparecer cinco naturalezas de conocimiento:

```
definition
requirement
constraint
decision
question
```

No como tipos de input.

Como tipos de conocimiento extraído.

La diferencia es esencial.

IASI no clasifica documentos.

Entiende conceptos.

Y después les da forma.

Fuentes

- `chatgpt-20260817130542-iasi-workflow-inputs-definitions.md`.
- `chatgpt-20260817135400-skill-define.md`.

title: “`define` no genera: reconcilia”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Una vez que aparecieron `definitions/`, parecía natural crear una operación capaz de producirlas.

La palabra fue:

```
define
```

Al principio podía imaginarse como un generador:

```
inputs
  ↓
define
  ↓
definitions
```

Pero esa imagen duró poco.

Las definitions no eran un artefacto derivado que pudiera borrarse y regenerarse cada vez.

Eran **estado canónico editable**.

Una persona podía revisar una definition, afinar una explicación, aclarar una relación o mejorar su representación. Si la siguiente ejecución de `define` machacaba esos cambios porque no estaban literalmente en el input, el sistema estaría destruyendo conocimiento.

Así que la responsabilidad cambió.

`define` debía **reconciliar**.

Su contexto real era algo parecido a:

```
inputs actuales
+
definitions existentes
+
refinamientos humanos
+
templates
        ↓
      define
        ↓
estado canónico coherente
```

Esto obligó a separar razonamiento y mecánica.

El skill debía entender:

- relaciones entre fuentes;
- conceptos equivalentes;
- contradicciones;
- información ausente;
- cuándo algo es requisito o restricción;
- cuándo una pregunta ha quedado resuelta;
- cuándo una definition debe dividirse o consolidarse.

El runtime debía ocuparse de lo determinista:

- descubrir ficheros;
- fingerprints;
- gates;
- históricos;
- aplicar operaciones;
- proteger escrituras;
- registrar estado.

La frontera terminó resumiéndose así:

```
skill      → entiende y clasifica
templates  → dan forma
runtime    → valida y ejecuta
```

También apareció una fase explícita que resultó muy útil:

```
Analyse
   ↓
Classify
   ↓
Reconcile
   ↓
Draft
```

`Classify` no clasifica inputs.

Clasifica el conocimiento que ya se ha comprendido.

Por ejemplo:

```
"El sistema debe generar HTML y PDF"
    → requirement

"Para websites solo se genera HTML"
    → constraint

"Hemos decidido usar Quarto"
    → decision

"No sabemos cómo resolver EPUB"
    → question
```

Después llegaron las templates.

No debían convertirse en formularios que forzaran la realidad a entrar en cinco cajas. Debían ofrecer formas canónicas cuando la naturaleza semántica del conocimiento encajara.

Y ahí apareció otra propiedad esencial de `define`: **idempotencia semántica**.

Si nada relevante ha cambiado, volver a ejecutar `define` no debería reorganizar el mundo por capricho.

Si una persona ha refinado una definition y los inputs no invalidan ese refinamiento, debe conservarse.

Si nueva evidencia resuelve una `question`, puede reclasificarse.

Si dos definitions resultan ser el mismo concepto, pueden consolidarse.

La operación ya no era "generar documentación".

Era mantener una base de conocimiento coherente a medida que cambia la evidencia.

Eso es una cosa bastante distinta.

Fuentes

- chatgpt-20260817135400-skill-define.md.

title: “IASI decide construirse con IASI”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

La frase apareció antes de que estuviéramos preparados para todas sus consecuencias:

> **IASI se construye con IASI.**

Hasta entonces IASI era la metodología que estábamos creando para trabajar sobre otros proyectos.

`iasi-graphics` había sido su primera PoC.

Pero llegó un punto en que mantener una excepción para el propio IASI empezaba a ser absurdo.

Si `inputs/`, `definitions/`, skills, commands, validaciones y gates eran una mejor forma de construir software, el primer proyecto que debía someterse a ellas era el propio sistema que las implementaba.

Eso introducía un problema clásico de bootstrapping.

El IASI existente había sido construido, en gran parte, antes de disponer del IASI que ahora queríamos utilizar.

No tenía sentido fingir lo contrario.

La solución conceptual fue separar dos estados.

El código actual podía actuar como **bootstrap**.

El siguiente IASI debía empezar a desarrollarse siguiendo IASI.

Para no perder el punto de partida, congelamos el estado existente con un tag.

Después apareció una rama:

```
self-hosting
```

La intención era clara:

```
IASI existente
    ↓
bootstrap
    ↓
IASI aplicado sobre sí mismo
    ↓
IASI siguiente
```

El código anterior no se convertía en basura.

Seguía siendo implementación, referencia, evidencia y fuente de conocimiento.

Pero dejaba de ser automáticamente la especificación del futuro.

Eso es una distinción importante.

Cuando un sistema se reconstruye a sí mismo existe una tentación enorme de describir como requisito todo aquello que ya hace.

Pero IASI había nacido, precisamente, para cuestionar las soluciones actuales.

La implementación bootstrap podía enseñar.

No podía mandar.

Los primeros inputs del self-hosting empezaron a describir el propio mecanismo de definitions y `define`.

La idea era atractiva porque convertía el desarrollo en una prueba continua:

> Si IASI no puede construir IASI, tenemos un problema bastante más interesante que un test fallido.

Sin embargo, casi inmediatamente descubrimos otra cosa.

La recursión conceptual era correcta.

Nuestra capacidad para mantenerla entera en la cabeza no lo era.

Fuentes

- `chatgpt-20260817130542-iasi-workflow-inputs-definitions.md`.
- Conversación de 2026-08-17 sobre el tag de bootstrap y la rama `self-hosting`.
- Memoria de trabajo de IASI.

title: “Nos perdimos y paramos”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

El self-hosting produjo entusiasmo durante unos minutos.

También produjo confusión.

Habíamos congelado el IASI existente con un tag. Habíamos creado la idea de una rama `self-hosting`. Teníamos inputs sobre `definitions` y el skill `define`. Empezamos a hablar de procesarlos, generar definitions, reconstruir el programa y usar el viejo IASI como bootstrap.

Todo era razonable por separado.

Junto, dejó de ser comprensible.

La frase que cortó la secuencia fue:

> **Me estoy perdiendo.**

Intentamos simplificar.

Enumeramos dónde estábamos.

Volvimos a explicar que todavía no habíamos recreado IASI.

Apareció la rama.

Volvimos a preguntar qué debía hacer Copilot.

Y finalmente:

> **Vamos a parar aquí.**

Ese momento merece una entrada del Journey precisamente porque no produjo código.

Produjo información sobre la metodología.

Los Sistemas Inteligentes eliminan muchísimas fricciones que antes actuaban como pausas naturales.

Una pregunta obtiene respuesta inmediatamente. Una arquitectura puede extenderse en segundos. Mientras un agente implementa una cosa, podemos abrir otra línea de pensamiento. Los bloqueos desaparecen y con ellos desaparecen también intervalos en los que el humano solía recuperar contexto.

La velocidad técnica aumenta.

La capacidad humana para comprender, decidir y asimilar no aumenta en la misma proporción.

Ese desfase ya había aparecido como principio metodológico de IASI:

No confundir disponibilidad de la IA con disponibilidad humana.

El episodio de self-hosting lo convirtió en experiencia directa.

No había un error técnico.

Había demasiados cambios de nivel conceptual seguidos:

```
bootstrap
→ tag
→ branch
→ inputs
→ definitions
→ skill
→ command
→ reconstrucción
→ Copilot
```

El sistema de ideas seguía avanzando.

El modelo mental humano empezaba a quedarse atrás.

La respuesta correcta no era pedir a la IA que explicara todavía más deprisa.

Era parar.

Esto introduce una clase de gate que no habíamos modelado en el workflow.

Tenemos gates técnicos:

```
validated
planned
verified
```

Pero puede existir otro límite:

```
¿el humano sigue entendiendo lo suficiente
como para tomar la siguiente decisión?
```

No es fácilmente automatizable y quizá nunca deba serlo.

Pero sí debe formar parte de la metodología.

La velocidad sostenible de un sistema humano-IA no es la velocidad máxima de los agentes.

Es la velocidad a la que el humano puede mantener comprensión suficiente para seguir siendo responsable de las decisiones.

Lo que quedó

Parar no interrumpió el proceso.

Fue parte del proceso.

Y el hecho de que esta entrada exista es exactamente la razón por la que necesitábamos un Journey.

El book futuro podrá explicar limpiamente el self-hosting.

El Journey debe conservar que, cuando llegamos por primera vez allí, nos hicimos un lío.

Fuentes

- Conversación de 2026-08-17 sobre self-hosting de IASI.
- Memoria de IASI sobre límites humanos, fatiga cognitiva y velocidad sostenible.

title: “El Journey empieza a escribirse”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

El Journey llevaba varios días existiendo como idea antes de tener documentos.

Eso también era coherente con su naturaleza.

Primero apareció la necesidad de conservar conversaciones. Después la distinción entre raw y treated. Luego la separación entre Journey, decisiones, libros y artefactos. Más tarde empezamos a señalar explícitamente frases y episodios como “esto es Journey”.

Pero todavía no lo habíamos escrito.

La pregunta que finalmente puso el repositorio en movimiento fue sencilla:

> ¿Te apetece empezar con el Journey?

La primera discusión no fue sobre Quarto.

Fue sobre el enfoque.

No queríamos un diario:

```
17 de agosto
hoy hicimos...
después hicimos...
```

La fecha importaba, pero no debía ser la unidad narrativa.

La unidad debía ser **el episodio que cambia algo**.

Una idea aparece, se discute, tropieza con la realidad y deja una consecuencia.

Eso permite que una entrada dure una hora o atraviese varios días.

También decidimos algo más importante que la estructura:

> No reescribir la historia para que parezca que sabíamos desde el principio dónde íbamos.

Si `definitions/` surgió después de varios intentos, el Journey debía contarlo así.

Si OpenSpec parecía necesario y después dejó de serlo, ambas cosas debían coexistir.

Si una propuesta vino primero de la IA y Javier la corrigió, eso también forma parte del proceso.

Si nos perdimos, se escribe que nos perdimos.

Después sí llegó Quarto.

El Journey seguirá siendo un proyecto capaz de producir HTML y PDF. La lectura principal puede ser viva y web, pero periódicamente podrá cristalizar en una edición imprimible.

Eso introduce una tensión interesante.

El contenido debe seguir siendo histórico e inmutable en su sentido.

La publicación, en cambio, puede evolucionar: estilo, navegación, índices, enlaces, selección editorial.

Como ocurría con raw y treated, volvemos a separar fuente y representación.

Y finalmente llegó el gesto que convierte la idea en proyecto:

entregar los chats acumulados y decir:

> Empieza a crear documentos de Journey. Todos los que quieras.

Esta primera tanda no pretende cerrar la historia.

Hace algo más útil.

Crea un primer mapa narrativo de lo que ya ocurrió y deja un formato con el que podamos seguir trabajando.

A partir de aquí, el Journey ya no es una promesa en otro repositorio.

Ha empezado.

Fuentes

- Conversación de 2026-08-17 sobre el enfoque y estructura de `iasi-journey`.
- Chats entregados para la primera construcción del Journey.
- Memoria de trabajo de IASI.

title: “Publicaciones multidocumento”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Durante la evolución de `iasi-quarto` apareció una necesidad que no habíamos identificado inicialmente: una publicación no tiene por qué corresponderse con un único documento ni con un único formato de salida.

El punto de partida era mucho más sencillo. IASI trabajaba fundamentalmente con dos formas de publicación:

- HTML, orientado a web.
- PDF, orientado a documento cerrado, distribución e impresión.

La arquitectura estaba construida alrededor de esa realidad.

Sin embargo, al comenzar a estudiar nuevos formatos de salida fueron apareciendo sucesivamente otras necesidades:

- EPUB para libros electrónicos.
- ODT para documentos editables y compatibilidad ofimática.
- GFM para publicación y reutilización directa en GitHub.
- Typst como alternativa moderna para la generación de PDF.

Lo que inicialmente parecía una simple ampliación de formatos terminó revelando un problema conceptual distinto.

No estábamos añadiendo únicamente nuevos formatos de exportación.

Estábamos descubriendo que **una misma publicación puede necesitar múltiples materializaciones simultáneas**.

El problema

Una aproximación sencilla habría sido considerar uno de los formatos como principal y convertir desde él al resto.

Por ejemplo:

```
HTML
     PDF
     EPUB
     ODT
```

Pero esta estructura introduce una dependencia artificial.

HTML no es necesariamente el origen conceptual de PDF.

PDF tampoco lo es de EPUB.

ODT no debería ser una conversión secundaria de ninguno de ellos.

Cada formato responde a un contexto de uso distinto y puede necesitar configuración, presentación y decisiones específicas.

Por tanto, el modelo más adecuado es:

```
contenido / publicación

           HTML
           PDF
           EPUB
           DOC / ODT
           Git / GFM
```

Todos son resultados de una misma fuente y de una misma definición de publicación.

Ninguno necesita ser considerado necesariamente el documento principal.

La necesidad descubierta

IASI necesita soportar **publicaciones multidocumento**.

Una publicación multidocumento es aquella que puede generar y distribuir varias materializaciones de un mismo contenido, manteniendo cada una:

- su identidad;

- su configuración;
- su proceso de generación;
- su estructura de salida;
- y su finalidad de uso.

La unidad lógica deja de ser exclusivamente un fichero.

La unidad lógica pasa a ser la **publicación**, formada potencialmente por varios documentos relacionados.

Por ejemplo:

```
_outputs/
  html/
  pdf/
  epub/
  doc/
  git/
```

Estos directorios no representan cinco publicaciones independientes.

Representan cinco materializaciones de una misma publicación.

Identidad de publicación frente a implementación

La aparición de varios formatos reveló además la necesidad de separar dos conceptos que inicialmente podían confundirse:

1. La identidad pública del formato.
2. La tecnología utilizada para producirlo.

Por ejemplo:

```
pdf
```

puede ser la identidad que entiende el usuario, aunque internamente Quarto utilice:

```
typst
```

De la misma forma:

```
doc
```

puede ser una identidad más natural dentro de IASI aunque actualmente su implementación utilice:

```
odt
```

Y:

```
git
```

puede representar una publicación destinada a GitHub mientras actualmente utiliza:

```
gfm
```

Esto conduce a una separación:

```
identidad pública    implementación actual

pdf                  typst
doc                  odt
git                  gfm
```

La implementación puede cambiar sin modificar necesariamente la identidad pública.

Compatibilidad directa con la tecnología subyacente

Al mismo tiempo, IASI no debe impedir que un usuario que conoce Quarto utilice directamente sus nombres nativos.

Por ello pueden coexistir:

```
doc
odt

git
gfm

pdf
typst
```

Cuando el usuario especifica explícitamente un formato, IASI debe respetar esa identidad.

Por ejemplo:

```
build(format = c("doc", "odt"))
```

puede producir ambas materializaciones si ambas están configuradas, incluso aunque actualmente compartan la misma tecnología subyacente.

Esto resulta importante porque una equivalencia actual no implica una equivalencia futura.

`doc` puede utilizar ODT hoy y convertirse más adelante en una publicación Word independiente.

La arquitectura no debería tener que cambiar cuando eso ocurra.

Perfiles independientes

Esta necesidad llevó también a otra decisión: cada identidad de publicación debe poder disponer de su propio perfil Quarto.

Por ejemplo:

```
_quarto-html.yml
_quarto-pdf.yml
_quarto-epub.yml
_quarto-doc.yml
_quarto-odt.yml
_quarto-git.yml
_quarto-gfm.yml
_quarto-typst.yml
```

Aunque dos perfiles utilicen actualmente la misma tecnología, siguen siendo configuraciones independientes.

Así:

```
doc
→ _quarto-doc.yml
→ implementación ODT
→ _outputs/doc/
```

mientras:

```
odt
→ _quarto-odt.yml
→ implementación ODT
→ _outputs/odt/
```

La identidad se conserva desde la petición hasta el artefacto generado.

`all` como selección editorial

La aparición de alias y formatos técnicos mostró también que `all` no debe significar literalmente todos los nombres que el motor conoce.

Por ejemplo, generar simultáneamente:

```
doc + odt
git + gfm
pdf + typst
```

no sería una selección especialmente útil por defecto.

`all` representa mejor la colección de formatos que IASI considera adecuada para una publicación general.

Actualmente:

```
html
pdf
epub
doc
git
```

Los nombres técnicos continúan disponibles cuando se solicitan explícitamente.

Así distinguimos entre:

```
all
→ selección amigable definida por IASI
```

y:

```
format = ...
→ selección explícita realizada por el usuario
```

Además, `all` funciona como descubrimiento de las publicaciones configuradas.

Solo se generan aquellas para las que exista su correspondiente perfil `_quarto-xxx.yml`.

La ausencia de un perfil dentro de `all` no constituye un error ni requiere aviso.

Simplemente significa que esa materialización no forma parte de esa publicación.

Cuando un formato se solicita explícitamente y falta su perfil, IASI sí debe indicarlo y omitir esa materialización.

Por ejemplo:

```
Ignorando 'odt' porque falta '_quarto-odt.yml'.
```

Build y Publish

La publicación multidocumento obligó también a revisar la diferencia entre `build` y `publish`.

`build` genera las distintas materializaciones:

```
build()
    ↓
_outputs/
  html/
  pdf/
  epub/
  doc/
  git/
```

`publish` no debe fusionarlas ni reinterpretarlas.

Su primera responsabilidad es conservar ese conjunto:

```
_outputs/
    ↓
publish/
```

manteniendo exactamente la misma estructura.

Por tanto:

`build` genera las materializaciones.
`publish` prepara el conjunto para su despliegue.

Esto permite que `publish/` sea directamente el artefacto desplegable de la publicación multidocumento.

HTML como punto de entrada

En una publicación que contiene HTML, este puede actuar como interfaz de acceso al resto de materializaciones.

Inicialmente los formatos descargables aparecían como herramientas independientes en la barra de navegación.

Conforme aumentó el número de formatos, este modelo dejó de resultar adecuado.

PDF, EPUB, documentos editables o versiones GitHub acabarían llenando la navegación de iconos independientes.

Apareció entonces otra consecuencia natural de la publicación multidocumento: disponer de una única entrada:

```
Export
```

que permita acceder a las distintas materializaciones disponibles.

Por ejemplo:

```
Export
  PDF
  EPUB
  DOC
  GitHub
```

Este menú no forma parte conceptualmente del contenido fuente.

Es una representación de los artefactos que realmente forman parte de la publicación desplegada.

Un punto de integración explícito

Para evitar que `publish` tenga que interpretar o modificar arbitrariamente la estructura HTML generada por Quarto, se plantea incluir un punto de integración explícito propiedad de IASI.

Por ejemplo:

```
<!-- IASI_EXPORT -->
...
<!-- /IASI_EXPORT -->
```

o un elemento equivalente identificable de forma inequívoca.

Quarto decide dónde se encuentra ese elemento dentro de la interfaz.

IASI controla su contenido durante `publish`.

La separación queda entonces:

```
HTML / Quarto
    → decide dónde aparece Export

IASI publish
    → decide qué materializaciones aparecen dentro
```

Esto evita manipular indiscriminadamente la navegación generada por Quarto y proporciona a IASI un espacio explícitamente diseñado para ser modificado durante la publicación.

Consecuencia metodológica

La necesidad descubierta no pertenece únicamente a `iasi-quarto`.

Es una cuestión más general.

Cuando el coste de producir distintas representaciones de un mismo contenido disminuye, resulta menos necesario elegir anticipadamente un único documento final.

Un mismo conocimiento puede necesitar distintas formas según quién vaya a consumirlo y para qué:

```
navegar       → HTML
leer/imprimir → PDF
e-reader      → EPUB
editar        → DOC / ODT
versionar     → Git / GFM
```

La decisión deja de ser:

> ¿En qué formato está el documento?

y pasa a ser:

> ¿Qué materializaciones necesita esta publicación?

Necesidad incorporada

A partir de esta experiencia consideramos necesario que IASI pueda tratar una publicación como un conjunto coherente de materializaciones independientes.

No se trata únicamente de soportar más formatos.

Se trata de reconocer que:

> **Una publicación puede ser multidocumento.**

Y, por tanto:

> **El artefacto lógico no tiene por qué ser un fichero; puede ser el conjunto coherente de sus distintas materializaciones.**

Esta necesidad fue descubierta durante la evolución real de `iasi-quarto`, al intentar ampliar una arquitectura inicialmente HTML/PDF hacia EPUB, documentos editables, GitHub y nuevos motores de generación de PDF.

No fue un requisito definido de antemano.

Apareció como consecuencia del uso y de la evolución del propio sistema.

Precisamente por eso debe formar parte del journey.

title: “Los labs empiezan a tomar forma”

Hasta este momento habíamos hablado de laboratorios como una forma práctica de experimentar con IASI, pero sin definir todavía con precisión qué papel debían ocupar dentro del proyecto. Al trabajar sobre el **Volumen II**, la estructura empezó a aclararse: los labs no serían un complemento del libro, sino el propio contenido del volumen. Dicho de otra manera, **los labs son el Volumen II**.

Esto cambia bastante la forma de pensar el contenido. Cada laboratorio pasa a ser una unidad de trabajo completa y, editorialmente, un capítulo del libro. Un lab puede estar compuesto internamente por varios documentos, pero para quien recorre el volumen representa una experiencia única.

```
Lab 01
Lab 02
Lab 03
...
```

Los primeros laboratorios tienen además una característica importante: no necesitan una infraestructura específica. Su objetivo es comenzar a trabajar con Sistemas Inteligentes, experimentar con prompts, contexto, instrucciones y otros conceptos básicos, y pueden realizarse prácticamente desde cualquier entorno.

Pero esto deja de ser cierto en algún momento. A medida que los laboratorios avanzan empiezan a necesitar herramientas, servicios, runtimes, contenedores y productos propios de IASI. Aparece entonces una frontera bastante natural dentro del volumen.

```
Labs 01-09
    ↓
pueden ejecutarse en un entorno cualquiera

Lab 10
    ↓
construye el entorno de laboratorios
```

```
Labs 11+
    ↓
pueden asumir ese entorno
```

Así nace **Lab 10: Montando la infraestructura**. Y con él aparece una cuestión que hasta ahora no habíamos tenido que resolver: ¿dónde viven realmente los labs?

Podríamos ejecutarlos sobre la misma máquina utilizada para desarrollar IASI, pero eso mezclaría dos mundos diferentes. La máquina de desarrollo contiene repositorios, herramientas internas, configuraciones y dependencias que existen precisamente porque estamos construyendo IASI. Un usuario de los labs no debería necesitar todo eso.

Queremos que utilice IASI como producto, no que un laboratorio funcione porque casualmente tiene al lado el código fuente de `iasi.quarto`, `iasi-lua` o cualquier otro componente. Esto nos lleva a separar claramente los dos entornos.

```
iasi-dev
    ↓
entorno donde construimos IASI

iasi-labs
    ↓
entorno donde usamos IASI
```

La separación no es solamente técnica. También es una forma de comprobar que nuestros propios productos son realmente utilizables fuera del entorno en el que fueron creados. Si algo solo funciona en `iasi-dev`, probablemente todavía no sea un producto.

A partir de aquí empiezan a aparecer nuevas decisiones. Necesitamos una máquina específica para los laboratorios y queremos que ese entorno sea reproducible, esté aislado del entorno de desarrollo y sea suficientemente parecido a lo que podría encontrar una persona que llega a IASI desde fuera. La opción natural es utilizar una **máquina virtual dedicada a los labs**.

La máquina virtual nos permite construir el entorno desde cero, repetir el proceso cuando sea necesario y conservar determinados estados mediante checkpoints. También nos permite experimentar sin convertir la máquina de desarrollo en un ecosistema de dependencias acumuladas que termine haciendo funcionar cosas por motivos que ya no somos capaces de explicar.

Aparece entonces otra decisión: cuál debe ser el sistema operativo de referencia. IASI no pretende depender de un único sistema operativo, pero los laboratorios necesitan una referencia concreta sobre la que podamos escribir instrucciones reproducibles y comprobar que funcionan. Por ahora, esa referencia será **Windows**.

No significa que IASI sea un proyecto Windows ni que los laboratorios solo puedan ejecutarse allí. Significa simplemente que necesitamos un camino por defecto. Si intentamos documentar

desde el principio cada operación para Windows, Linux, macOS y cualquier otra combinación posible, el laboratorio termina explicando sistemas operativos en lugar de explicar aquello que queremos experimentar.

Con todo esto empieza a tomar forma algo bastante mayor que una simple colección de ejercicios. Porque no basta con escribir laboratorios. También tenemos que decidir **cómo gestionar los labs**.

Un laboratorio puede necesitar determinado software instalado, mientras que otro puede depender del resultado de un laboratorio anterior. Algunos deberían poder ejecutarse de manera independiente y otros podrían necesitar un estado concreto de la máquina. Puede ser necesario volver a un checkpoint, comprobar que determinadas dependencias están disponibles antes de empezar o validar que el entorno ha quedado en el estado esperado al terminar.

También tendremos que decidir qué ocurre con los artefactos generados durante cada ejercicio. Un lab puede crear código, repositorios, configuraciones, contenedores o servicios que sobrevivan al propio ejercicio. Eso obliga a preguntarnos qué puede asumir un lab al comenzar, qué dependencias puede tener respecto a otros, qué estado debe dejar al terminar y cómo podemos repetirlo sin depender accidentalmente de todo lo que ocurrió antes.

La cuestión de los checkpoints forma parte del mismo problema. Tenemos que decidir cuándo representan un estado estable de la infraestructura, cuándo son simplemente una ayuda para repetir un ejercicio y hasta qué punto deben formar parte del diseño formal de los laboratorios. Lo mismo ocurre con la instalación y actualización de los productos IASI: un usuario de los labs debería consumirlos como productos, no trabajar con sus repositorios fuente.

Todavía no tenemos respuesta para todas estas cuestiones, y precisamente por eso merece la pena registrarlas ahora. El Journey no existe únicamente para documentar decisiones terminadas. También debe conservar el momento en el que descubrimos un problema y todavía estamos intentando entender qué forma tiene.

Empezamos pensando en escribir algunos laboratorios. Después descubrimos que los labs constituían un volumen completo. Al avanzar apareció la necesidad de construir una infraestructura específica para ejecutarlos y, al intentar hacerlo de forma reproducible, apareció una nueva capa que no habíamos previsto inicialmente: la gestión de los propios laboratorios.

En otras palabras, empezamos escribiendo labs y hemos terminado descubriendo que también necesitamos hacer **ingeniería de laboratorios**.

title: “Congelar también es ingeniería”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Durante años actualizar tecnología había parecido una decisión casi automática.

Aparece una versión nueva, corrige problemas, incorpora capacidades y, en teoría, mejora la anterior. En un proyecto que trabaja precisamente con tecnologías que cambian a gran velocidad, la tentación de ir siempre detrás de la última versión es todavía mayor.

Pero al revisar el entorno de IASI apareció una pregunta bastante menos vistosa:

> ¿Qué ganamos realmente cambiando ahora?

La respuesta no siempre era suficiente para justificar el movimiento.

Una versión mayor de Java, una nueva versión de Quarto, un cambio importante en una librería o una actualización de una herramienta de agentes pueden parecer decisiones locales. En realidad pueden afectar a compatibilidad, dependencias, arquitectura, pruebas, documentación, despliegue y planificación.

Eso nos llevó a formular una regla que inicialmente parece conservadora y, vista desde ingeniería, resulta bastante razonable:

> **Las versiones críticas de un proyecto deben congelarse mientras no exista una necesidad que justifique cambiarlas.**

Congelar no significa negarse a evolucionar.

Significa que la novedad no constituye por sí sola un requisito.

```
versión nueva disponible
        ↓
¿resuelve una necesidad real?
        ↓
     sí / no
```

Si la respuesta es no, el valor de la estabilidad puede ser mayor que el valor de actualizar.

La idea encaja con un principio más general de IASI. No empezamos por la herramienta ni por lo que acaba de aparecer. Empezamos por el problema.

La misma lógica que aplicamos a una metodología también debe aplicarse a una versión tecnológica.

Esto adquiere todavía más importancia cuando los Sistemas Inteligentes reducen mucho el coste aparente de modificar software. Que un agente pueda actualizar rápidamente código, configuración o dependencias no significa que el coste global de la decisión haya desaparecido. Sigue existiendo el coste de comprender el impacto y, sobre todo, el riesgo de introducir cambio sin necesidad.

Lo que quedó

Actualizar es una operación técnica.

Decidir actualizar es una decisión de ingeniería.

Y, como cualquier decisión, necesita una razón mejor que «hay una versión nueva».

Fuentes

- Conversación de 2026-08-21 sobre congelación de versiones críticas durante un proyecto.
- Memoria de trabajo de IASI sobre estabilidad tecnológica y cambios de versiones mayores.

title: “Las reglas se vuelven artefactos”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Las instructions habían permitido separar reglas generales de los prompts concretos.

Pero todavía quedaba una pregunta incómoda:

> ¿qué aspecto tiene una regla cuando deja de ser una frase perdida dentro de una instruction?

La respuesta terminó siendo mucho más literal de lo esperado.

Una regla debía ser un artefacto propio.

No una línea enterrada en un prompt gigantesco. No una convención que solo existe en la cabeza de quien trabaja. No un bloque copiado a distintos agentes.

Un documento.

```
builtin/
  rules/
      coding/
      docs/
```

Y cada regla tendría una estructura normalizada:

```
Title
Metadata
Rule
Exceptions
Rationale
Sources
Examples
```

El detalle de `Examples` fue especialmente revelador. No todas las reglas necesitan ejemplos, pero el formato debía permitir añadir casos positivos, negativos o situaciones de aplicación cuando ayudaran a entender la norma.

La consecuencia importante no era editorial.

Al convertir las reglas en documentos independientes podían ser:

- versionadas;
- revisadas;
- trazadas;
- seleccionadas;
- combinadas;
- instaladas;
- procesadas por agentes;
- y discutidas una a una.

Después apareció el problema de la especificidad.

Una regla genérica de código puede decir algo razonable para la mayoría de lenguajes y, sin embargo, ser incorrecta para uno concreto.

El ejemplo terminó siendo muy claro.

La regla genérica de IASI establecía inicialmente un máximo de 150 caracteres por línea. Python podía imponer 99. Go, en cambio, podía indicar explícitamente que no debía aplicarse un límite fijo y que `gofmt` y la legibilidad eran la autoridad.

```
coding/styles/generic/*
          ↓
   regla general

coding/styles/python/*
          ↓
   regla específica
```

Cuando ambas regulan el mismo aspecto, la específica debe prevalecer.

> **La regla más específica tiene prioridad sobre la genérica.**

Eso evita dos extremos igual de malos: duplicar todas las reglas para cada lenguaje o fingir que una regla universal sirve siempre.

La biblioteca empezó a crecer con reglas genéricas y reglas específicas para Go, Java, Python, Bash y documentación. Pero el descubrimiento importante no fue la cantidad.

Fue que el comportamiento esperado del agente empezaba a existir como conocimiento estructurado fuera del agente.

Lo que quedó

Una regla IASI no es texto auxiliar para mejorar un prompt.

Es un artefacto canónico de ingeniería.

Y las reglas pueden formar una jerarquía donde el contexto más específico refine, e incluso sustituya, el comportamiento general.

Fuentes

- Biblioteca `builtin/rules/` desarrollada durante los días 26–28 de agosto de 2026.
- `builtin/rules/TEMPLATE.md`.
- Reglas `coding.styles.generic.maximum-line-length`, `coding.styles.python.maximum-line-length` y `coding.styles.go.no-fixed-line-length`.
- Conversaciones sobre normalización de rules y prioridad de reglas específicas sobre genéricas.

title: “Lo que IASI lleva consigo”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Cuando empezaron a existir rules, commands, definitions, configuración y plantillas apareció una pregunta muy práctica:

> ¿dónde vive todo eso cuando instalamos IASI?

Hasta entonces había sido fácil pensar en esos elementos como ficheros del repositorio con el que estábamos trabajando.

Pero IASI debía poder instalarse en otro proyecto sin depender de tener al lado el repositorio fuente completo.

La palabra que terminó ordenando el problema fue:

```
builtin/
```

`builtin/` representa aquello que IASI lleva **de fábrica**.

No es el estado de un proyecto concreto. No son inputs recibidos por ese proyecto. Es contenido canónico de la propia distribución: rules, definitions, configuration, templates, commands y otras piezas compartidas que puedan aparecer.

Eso produjo una separación importante entre fuente y distribución.

```
iasi-core
   ↓
builtin/
   ↓
construcción
   ↓
iasi-cli
   ↓
ejecutable Go con builtin embebido
```

`iasi-core` podía conservar la definición canónica. El ejecutable podía llevar una copia sincronizada y embebida durante el build, de manera que instalar IASI no requiriera buscar esos recursos por rutas externas.

La idea era exactamente la misma que habíamos aplicado otras veces: la complejidad puede estar dentro, pero no debe trasladarse al usuario.

En paralelo las tasks dejaron de ser simples documentos que «alguien ejecutará» y empezaron a adquirir un ciclo de vida explícito.

Aparecieron operaciones como:

```
task.start
task.run
task.done
task.rerun
task.reopen
```

y estados conceptuales como:

```
pending
   ↓
active
   ↓
done/success
done/failed
done/partial
done/cancelled
```

También apareció una distinción especialmente útil:

```
done != validated
```

Terminar una tarea no demuestra automáticamente que el resultado sea correcto.

Otra pieza que se fue haciendo más concreta fue la frontera entre la parte determinista y la parte agentic. Los agentes pueden interpretar y ejecutar trabajo, pero gates, estado, seguridad de filesystem y otras operaciones mecánicas no deberían reimplementarse de forma distinta para Copilot, Codex o cualquier plataforma futura.

Por eso la plataforma debía terminar llegando al mismo runtime IASI.

```
Copilot / Codex / otra plataforma
              ↓
           adapter
              ↓
      comando canónico IASI
              ↓
        runtime compartido
```

Lo que unas semanas antes era una intención arquitectónica empezaba a tener ficheros, código, comandos y build.

Lo que quedó

IASI ya no era únicamente una metodología explicada en documentos.

Empezaba a tener un **contenido propio que podía transportar**, un runtime común y una forma explícita de representar trabajo.

El sistema empezaba a parecerse a lo que más tarde llamaríamos con mucha naturalidad el **ENGINE**.

Fuentes

- `go-builtin-build.zip`.
- `iasi-task-commands-v4.zip`.
- `runtime-interface.md`.
- Estructura de `iasi-core`, `iasi-cli` e `iasi-ai` consolidada durante los días 28–29 de agosto de 2026.

title: “El código es un output”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Durante mucho tiempo, incluso hablando de IASI, resultaba natural imaginar el flujo de ingeniería con el código al final.

La costumbre pesa.

En ingeniería de software solemos hablar de requisitos, diseño e implementación como una secuencia que desemboca en código. La documentación, los diagramas, los tests, la configuración o la infraestructura aparecen alrededor de ese centro.

Pero mientras intentábamos explicar IASI de la forma más simple posible apareció otra imagen:

```
INPUTS → ENGINE → OUTPUTS
```

Y entonces una observación cambió el dibujo entero:

> **El código es un output.**

No *el* output.

Un output.

El mismo ENGINE puede producir, según la necesidad:

```
OUTPUTS
  code
  tests
  documentation
  books
  presentations
  diagrams
  configuration
  infrastructure
  ...
```

La idea conecta con algo que ya habíamos descubierto en publicación multidocumento. Una misma fuente podía tener varias materializaciones sin que una de ellas tuviera que ser «la principal».

Ahora el concepto se ampliaba mucho más allá de una publicación.

El trabajo de ingeniería transforma conocimiento en artefactos.

A veces el artefacto adecuado es código.

A veces no.

Y muchas veces son varios a la vez.

Esto cambia también el lugar conceptual de la ingeniería dentro de IASI.

Si el flujo fuera:

```
inputs → code
```

la metodología terminaría inevitablemente orientada a generar software.

Al introducir el ENGINE aparece otra cosa:

```
INPUTS
   ↓
ENGINE
   ↓
OUTPUTS
```

El centro ya no es el lenguaje de programación ni la herramienta que genera el resultado.

El centro es la transformación: comprender lo que entra, aplicar conocimiento, reglas, decisiones, tareas y capacidades, y producir aquello que la necesidad exige.

Eso permite que un libro, una infraestructura Docker, una presentación HTML o una modificación de código sean tratados como consecuencias distintas del mismo proceso de ingeniería.

También elimina una jerarquía que ya empezaba a resultar artificial.

La documentación no es «lo que acompaña al código».

Un diagrama no es «un extra».

Una presentación no es «otra cosa que hacemos después».

Todos pueden ser outputs de primera clase.

Lo que quedó

La fórmula que apareció casi como una slide terminó funcionando como modelo general:

```
INPUTS → ENGINE → OUTPUTS
```

Y con ella el código perdió un privilegio que nunca necesitó.

IASI no existe para producir código. Existe para hacer ingeniería.

Fuentes

- Conversaciones del 30 de agosto de 2026 durante el diseño de `Introducing IASI`.
- `inputs/externals/001-first.md`.
- Necesidad de publicación multidocumento registrada en el Journey.
- Memoria de trabajo de IASI: «el código es un output».

title: “El prompt deja de ser el contenedor”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

IASI había empezado, como casi todo el mundo, utilizando prompts para proporcionar instrucciones y contexto.

Después empezamos a sacar de ellos las instructions.

Luego las skills.

Después las rules.

Y finalmente apareció una pregunta bastante obvia una vez formulada:

> Si una decisión ya existe, ¿por qué seguimos confiando en que permanezca dentro de una conversación o de un prompt?

La respuesta llevó a un principio mucho más radical de lo que parece.

El prompt no debe ser el contenedor principal del conocimiento.

La conversación es magnífica para pensar.

Podemos discutir una posibilidad, llevarla al extremo, descartarla, recuperarla, hacer bromas, detectar contradicciones y terminar tomando una decisión.

Pero la conversación no debería ser el único lugar donde esa decisión existe.

Cuando algo importa, debe persistirse como artefacto.

Incluso cuando el artefacto parece ridículamente pequeño.

Una sola línea escrita en un documento puede costar prácticamente lo mismo que escribirla en el chat. Sin embargo, el documento convierte esa línea en algo:

- persistente;
- versionable;
- trazable;
- revisable;

- reutilizable;
- direccionable;
- y procesable por otros agentes.

El cambio conceptual puede representarse así:

```
antes

prompt = petición + reglas + conocimiento + decisiones + contexto
```

frente a:

```
después

artefactos = conocimiento persistente
prompt     = activación / petición mínima
```

Esto no elimina el prompt.

Lo pone en su sitio.

Un agente puede terminar recibiendo algo tan pequeño como:

```
Execute engine/tasks/001-first.md
```

porque el resto ya existe en el workspace.

El conocimiento no necesita volver a ser narrado cada vez.

Este principio también cambia la relación entre humano y agente. La memoria operativa del proyecto deja de depender de que el modelo «recuerde» una conversación anterior. Otro agente puede llegar después, leer los mismos artefactos y continuar desde el mismo estado explícito.

Por eso la persistencia documental no es simplemente una preferencia de documentación.

Es una condición para que el ENGINE pueda ser independiente de una conversación concreta y de un proveedor concreto.

Lo que quedó

El chat sirve para pensar.

Los artefactos sirven para conservar lo que hemos aprendido y decidido.

> **Pensar puede ser conversacional. El conocimiento operativo debe poder sobrevivir a la conversación.**

Fuentes

- Conversación del 30 de agosto de 2026 sobre prompts, documentos y persistencia del conocimiento.
- Experimento `introducing-iasi`, donde el trabajo pasó de un prompt directo a `inputs/` y `engine/tasks/`.
- Principios previos de IASI sobre inputs inmutables, definitions canónicas y trazabilidad.

title: “La infraestructura sale de la máquina de desarrollo”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Los labs ya nos habían obligado a separar la máquina donde construimos IASI de la máquina donde se utiliza IASI.

Poco después apareció una separación distinta.

La máquina de desarrollo tampoco tenía por qué ejecutar todos los servicios que el entorno necesita.

Docker, PlantUML, Ollama y Open WebUI podían formar una capa de servicios independiente.

La idea era sencilla sobre el papel:

```
iasi-dev
   ↓ consume

iasi-docker
   PlantUML
   Ollama
   Open WebUI
```

En la práctica nos obligó a comprobar varias cosas que una instalación «todo en localhost» oculta con mucha facilidad.

Primero, `localhost` no es una propiedad del servicio.

Es una propiedad del consumidor.

Si PlantUML vive en otra máquina, la definición funcional del servicio no cambia. Cambia la dirección desde la que lo consumimos.

```
misma máquina
    → localhost:10xxx

otra máquina
    → host-servicios:10xxx
```

Segundo, la red interna de Docker y la red desde la que acceden otros consumidores son cosas diferentes. Open WebUI puede hablar con Ollama mediante `http://ollama:11434` dentro de la red Docker mientras un cliente externo utiliza el puerto publicado por la máquina de servicios.

Tercero, la persistencia debía pertenecer a los servicios y no al ciclo de vida de un contenedor. Los modelos de Ollama y los datos de Open WebUI debían sobrevivir a recreaciones de los contenedores.

Y finalmente apareció la GPU.

La máquina `iasi-docker` terminó utilizando la NVIDIA disponible para acelerar Ollama, mientras el host conservaba su propia capacidad gráfica. Llegar hasta ahí exigió resolver Hyper-V, GPU-P, drivers, NVIDIA Container Toolkit y varios detalles bastante menos elegantes que el diagrama final.

Precisamente por eso la separación merece la pena.

Una vez resuelta, la complejidad queda encapsulada detrás de servicios estables.

La máquina de desarrollo no necesita saber cómo está construido Ollama internamente ni cómo se expone la GPU al contenedor. Necesita una capacidad accesible mediante un contrato conocido.

Esto volvió a repetir un patrón que IASI ya había encontrado en otros sitios:

La definición pertenece al sistema. La ubicación física puede cambiar.

Lo que quedó

IASI empezó a distinguir con más claridad entre:

```
entorno de desarrollo
capacidad de infraestructura
proyecto consumidor
```

No todo tiene que vivir en la misma máquina para formar parte del mismo proceso.

Y una infraestructura bien diseñada debería poder moverse sin obligar a redefinir aquello que proporciona.

Fuentes

- `20-docker.qmd` y versiones previas de la definición Docker de IASI.
- `iasi-compose.yml` y revisiones del 29–30 de agosto de 2026.
- Sesiones de configuración de `iasi-docker` con GPU NVIDIA, Ollama, Open WebUI y PlantUML.
- `iasi-models.md`.

title: “Descargamos modelos. Construimos agentes.”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Cuando Ollama empezó a funcionar con GPU apareció una satisfacción bastante inmediata: ya teníamos modelos locales.

Qwen, Llama, Phi, Gemma, DeepSeek.

Era fácil describir aquello como «ya tenemos agentes locales».

Pero no era verdad.

Habíamos descargado modelos.

Un modelo proporciona capacidad de inferencia. Puede razonar, escribir, resumir, programar o analizar según sus capacidades. Pero no conoce por sí solo qué significa trabajar dentro de IASI.

La distinción terminó formulándose de una manera mucho más útil:

> **Descargamos modelos. Construimos agentes.**

Un agente IASI necesita algo más parecido a esto:

```
agente
  modelo
  rules
  definitions
  tools
  context
  workflow
```

El modelo es una pieza.

No la arquitectura completa.

Esto también modificó la pregunta de selección.

No necesitamos encontrar «el mejor modelo» en abstracto.

Necesitamos elegir **el modelo suficiente para la tarea**.

Un modelo pequeño y rápido puede ser mejor para clasificación, transformación o tareas rutinarias. Otro puede ser más adecuado para razonamiento. Uno multimodal puede cubrir imágenes. Los modelos mayores pueden reservarse para trabajo que realmente necesite esa capacidad.

La GPU disponible reforzó esta idea. Con 6 GB de VRAM no tenía sentido convertir cada tarea en una competición por ejecutar el modelo más grande posible. Los modelos de 3B–4B proporcionaban un punto especialmente útil entre velocidad, capacidad y memoria; otros más grandes podían utilizarse cuando la tarea lo justificara.

Y entonces apareció otra consecuencia que ese mismo día dejaría de ser teórica.

Si las tasks, rules, definitions y outputs están fuera del modelo, **el ejecutor puede cambiar**.

Podemos utilizar un modelo local para determinado trabajo, Codex para otro, ChatGPT para otro o contrastar varios sistemas sobre la misma tarea.

El agente deja de ser una caja indivisible ligada a un proveedor.

Lo que quedó

Ollama no nos dio «IASI local».

Nos dio una pieza fundamental: capacidad de inferencia local que podemos incorporar al ENGINE.

Eso permite construir agentes sin asumir que todo el proceso depende de una cuota externa o de un único servicio.

La unidad de diseño deja de ser «qué IA usamos».

Pasa a ser:

> **qué capacidad necesita esta tarea y con qué combinación de modelo, contexto, reglas, herramientas y workflow la proporcionamos.**

Fuentes

- `iasi-models.md`.
- Stack local de Ollama configurado el 30 de agosto de 2026.
- Conversación de 2026-08-30 sobre la diferencia entre modelos y agentes.
- Principios previos de IASI sobre trabajo con múltiples Sistemas Inteligentes.

title: “Una presentación nos obliga a aplicar IASI”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Queríamos presentar IASI.

La reacción inicial fue completamente normal.

Pensamos en presentaciones.

PowerPoint. Gamma. Prezi. HTML. Herramientas capaces de generar slides con ayuda de agentes.

Y durante unos minutos empezamos a comparar soluciones.

Hasta que apareció una objeción bastante incómoda:

> Estamos rompiendo el primer principio de IASI.

Habíamos empezado por las herramientas disponibles y estábamos intentando encajar el problema dentro de ellas.

Exactamente aquello que llevábamos semanas diciendo que no había que hacer.

Así que volvimos atrás.

La pregunta dejó de ser:

> ¿Con qué hacemos la presentación?

Y pasó a ser:

> ¿Qué necesitamos que sea esta presentación?

La respuesta empezó a dibujar una experiencia sencilla, progresiva, visual e interactiva. Algo que pudiera seguir siendo una presentación y, al mismo tiempo, permitiera entrar y salir de conceptos sin quedar limitado por el modelo clásico de diapositiva estática.

Solo después apareció HTMLSlides como una posibilidad interesante.

No porque «tocara usar HTML».

Porque respondía razonablemente a la necesidad que acabábamos de describir.

El primer experimento fue deliberadamente pequeño:

```
Introducing IASI
Intelligent Assisted Software Engineering
```

Funcionó.

Y entonces hicimos algo todavía más importante: dejamos de trabajar mediante un prompt improvisado y empezamos a organizar el propio experimento como IASI.

```
inputs/
engine/
output/
```

El input externo quedó en:

```
inputs/externals/001-first.md
```

La decisión de ejecución terminó convertida en:

```
engine/tasks/001-first.md
```

Mantener `001-first` en ambos lados resultó útil: la misma unidad de trabajo podía conservar una identidad reconocible mientras cambiaba de papel dentro del proceso.

Y el HTML generado era un output:

```
output/introducing-iasi.html
```

La propia estructura del repositorio empezó a representar el modelo que queríamos explicar.

```
INPUTS → ENGINE → OUTPUTS
```

Entonces ocurrió algo que no habíamos planeado.

Codex alcanzó el límite de uso justo cuando íbamos a ejecutar la task.

El proceso se detuvo.

Durante unos segundos parecía simplemente una incidencia molesta.

Pero la task existía. El input existía. El HTML existente existía. El conocimiento necesario para continuar no estaba encerrado en la sesión de Codex.

Otro agente pudo leer esos mismos artefactos y ejecutar la misma tarea sobre el HTMLSlides ya generado.

Y funcionó.

La anécdota dejó una prueba bastante más valiosa que la slide:

> **La task describe el trabajo. El agente es un posible ejecutor.**

Codex podía quedarse sin cuota sin que la definición del trabajo desapareciera con él.

Aquello conectó de golpe varias decisiones que habíamos ido tomando por separado: artefactos persistentes, tasks, adapters, runtime común, modelos locales y outputs independientes del ejecutor.

Lo que quedó

La primera presentación de IASI terminó siendo también un pequeño ensayo de IASI.

Empezamos incumpliendo nuestro propio principio al pensar primero en herramientas.

Lo detectamos.

Volvimos a la necesidad.

Elegimos una tecnología después.

Persistimos input y task.

Un agente falló por disponibilidad.

Otro pudo continuar.

Para una primera slide, no estuvo mal.

Fuentes

- Conversaciones del 30 de agosto de 2026 sobre el diseño de `Introducing IASI`.
- `inputs/externals/001-first.md`.
- `engine/tasks/001-first.md`.
- `output/introducing-iasi.html` y versión de tres slides generada durante el experimento.
- HTMLSlides v0.9.4 utilizado como skill de ejecución.

title: “Un lenguaje visual para IASI”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

El primer HTMLSlides salió sorprendentemente bien.

También salió con una estética que había decidido HTMLSlides.

Eso era útil para validar la herramienta, pero introducía un problema distinto.

Si mañana hacíamos otra presentación con PowerPoint, un diagrama con PlantUML y una página web con Quarto, cada tecnología podía terminar hablando un idioma visual diferente.

Y entonces apareció la pregunta correcta:

> ¿Por qué debería ser HTMLSlides quien decida cómo se ve IASI?

La respuesta fue inmediata.

No debería.

El lenguaje visual pertenece a IASI.

HTMLSlides, PowerPoint, PlantUML, Quarto o cualquier otra tecnología solo deben aprender a expresarlo.

La diferencia se parece mucho a la que ya habíamos descubierto con las plataformas agentic.

No queremos:

```
HTMLSlides → define estilo
PowerPoint  → define otro
PlantUML    → define otro
```

Queremos:

```
IASI Visual Language
        ↓
   implementaciones
      HTMLSlides
      PowerPoint
      PlantUML
      Quarto / web
      ...
```

Ese lenguaje puede incluir:

- tipografía;
- paleta;
- jerarquía;
- formas;
- líneas;
- cajas;
- iconografía;
- espaciado;
- composición;
- tratamiento de diagramas;
- movimiento cuando exista;
- y reglas de uso de todos esos elementos.

Algunas decisiones serán universales. Otras necesitarán una adaptación específica según las capacidades del medio.

PlantUML no expresa una composición exactamente igual que PowerPoint. Un documento impreso no utiliza movimiento. Una slide puede necesitar relaciones espaciales que no tienen sentido en una página de documentación.

Pero la identidad no debería reconstruirse desde cero cada vez.

Aquí apareció otra vez el patrón de adapters.

La definición canónica está en IASI.

Cada tecnología proyecta esa definición a su propio lenguaje.

Esto convierte lo visual en algo más que branding.

Pasa a ser otra parte gobernable y reutilizable de la ingeniería de artefactos.

Lo que quedó

`Introducing IASI` no debe definir la identidad visual de IASI.

Debe **consumirla**.

Y lo mismo harán los libros, los diagramas, las presentaciones futuras y los productos que necesiten representación visual.

La estética deja de pertenecer a la herramienta que renderiza.

Pasa a ser conocimiento canónico del sistema.

Fuentes

- Conversación del 30 de agosto de 2026 tras la primera prueba con HTMLSlides.
- Experiencia previa de estilo compartido en `iasi-home` y publicaciones Quarto.
- Uso de PlantUML e `iasi-graphics` como consumidores visuales dentro del ecosistema.

title: “La frontera también se protege”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Estábamos pensando en la siguiente slide de `Introducing IASI`.

La primera mostraba el modelo mínimo:

```
INPUTS → ENGINE → OUTPUTS
```

La siguiente debía enseñar algo más interesante: la realimentación.

Los inputs no nacen únicamente fuera del sistema. El propio trabajo del engine y la observación de los outputs pueden descubrir nueva información, nuevas restricciones o nuevas necesidades que vuelven a alimentar el proceso.

La primera representación fue aproximadamente:

```
INPUTS
← fuentes externas
← ENGINE
← OUTPUTS

ENGINE
← INPUTS
← ENGINE
← OUTPUTS

OUTPUTS
← ENGINE
```

Entonces apareció una corrección importante.

Los outputs pueden provocar nuevos inputs, pero eso **no significa que puedan modificarse**.

Una vez materializados, los outputs son inmutables.

La observación de un output puede descubrir algo nuevo:

```
OUTPUT
   ↓ observar
nuevo conocimiento / problema / necesidad
   ↓
INPUT
   ↓
ENGINE
   ↓
nuevo OUTPUT
```

pero nunca:

```
OUTPUT
   ↓
editar el OUTPUT
```

Ese matiz nos llevó directamente a revisar el otro extremo del sistema.

Hasta ese momento hablábamos con frecuencia de `inputs/externals/` como la zona de material externo de IASI. Pero la slide hizo visible que estábamos mezclando dos conceptos diferentes:

- el exterior;
- y la copia que IASI conserva de algo que nació en ese exterior.

El exterior no es una carpeta.

Está fuera del sistema.

La relación correcta es:

```
OUTSIDE

     copy / import

inputs/externals/


INPUTS
```

`inputs/externals/` ya vive físicamente dentro del proyecto IASI.

Lo que lo convierte en *external* no es su ubicación, sino su **procedencia**.

Eso permitió formular una distinción sencilla:

Todo external es input, pero no todo input es external.

`inputs/externals/` solo recibe material procedente del exterior.

`INPUTS`, en cambio, puede incorporar conocimiento que proceda de distintas fuentes, incluida la actividad del propio sistema.

La slide estaba añadiendo otra capa al modelo.

Pero entonces apareció una consecuencia todavía más práctica.

Si `inputs/externals/` existe precisamente para conservar lo que IASI recibió desde fuera, limitarse a decir que sus archivos son "de solo lectura" resulta insuficiente.

Una copia puede ser sobrescrita accidentalmente.

Y si eso ocurre, la historia cambia.

Un requisito recibido el lunes podría ser sustituido el martes por una versión corregida y, al mirar el repositorio, parecería que IASI siempre recibió la versión del martes.

Eso destruiría precisamente la evidencia que queríamos conservar.

La inmutabilidad necesitaba dejar de ser únicamente una disciplina humana.

Empezaba a pedir mecanismo.

La incorporación de un external debería poder comportarse como una operación **create-only**:

```
OUTSIDE
   ↓ import
inputs/externals/<artifact>
```

Si el artefacto ya existe, una nueva incorporación no debería sobrescribirlo silenciosamente.

Si la fuente externa cambia, IASI debería incorporar otra evidencia, no reescribir la anterior.

La implementación concreta queda todavía abierta, pero aparecieron posibilidades naturales:

- impedir overwrites durante la importación;
- registrar una huella o hash del material recibido;
- detectar cambios posteriores;
- conservar identidad y procedencia;
- representar una nueva versión externa como un nuevo artefacto cuando sea necesario.

No decidimos todavía el mecanismo final.

Lo importante fue descubrir por qué hace falta.

Una simetría en los extremos

El modelo empezó a mostrar una propiedad interesante:

```
OUTSIDE


inputs/externals/   →   ENGINE   →   outputs/
   inmutable                         inmutable
```

Los dos extremos conservan evidencia.

En un extremo, evidencia de **qué recibió IASI**.

En el otro, evidencia de **qué produjo IASI**.

El trabajo ocurre en medio.

Y cuando algo observado en cualquiera de esos extremos obliga a cambiar el sistema, no corregimos el pasado.

Creamos nueva información y volvemos a recorrer el proceso.

Eso es especialmente coherente con el Journey.

La historia tampoco se limpia.

Lo que apareció

Una conversación sobre una slide terminó refinando varias ideas del modelo:

```
OUTSIDE
   ↓
inputs/externals/
   ↓
INPUTS   ENGINE → OUTPUTS
```

con tres distinciones que antes estaban menos visibles:

1. `OUTSIDE` es el origen externo, no una carpeta interna.
2. `inputs/externals/` es la copia fiel que IASI conserva de algo recibido desde fuera.
3. Una copia fiel necesita protección contra sobrescrituras si queremos que la inmutabilidad sea una propiedad real y no solo una recomendación.

Todavía no hemos decidido cómo se implementará esa protección.

Pero ya sabemos que la necesitamos.

Y otra vez la secuencia fue la misma:

```
idea metodológica
      ↓
necesidad descubierta
      ↓
 futura capacidad técnica
```

Fuentes

- Conversación del 30 de agosto de 2026 durante el diseño de las slides de `Introducing IASI`.
- Principio previo de inmutabilidad de `inputs/externals/`.
- Modelo emergente `INPUTS → ENGINE → OUTPUTS`.

title: “La documentación también tiene ciclo de vida”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Durante mucho tiempo habíamos tratado la documentación como un output más del sistema.

Eso ya era un avance importante.

Un User Guide, un Technical Guide o una referencia no eran materiales secundarios que aparecían después del software. Podían ser artefactos de primera clase, construidos a partir del mismo conocimiento del proyecto.

Pero al revisar cómo debía mantenerse un User Guide apareció un problema nuevo.

No todo el documento tiene el mismo origen.

Hay partes que IASI puede reconstruir con bastante precisión porque dependen de información explícita del proyecto:

```
producto
  ↓
comandos
configuración
estructura
capacidades
interfaces
  ↓
contenido derivable
```

Y hay otras partes que existen porque un autor decide explicar algo de una determinada manera:

- una introducción;
- una explicación pedagógica;
- una advertencia basada en experiencia;

- una comparación;
- una recomendación;
- un ejemplo especialmente útil;
- un apartado de Tips & Tricks.

Intentar regenerar ambos tipos de contenido con la misma política era peligroso.

Si IASI trataba todo el manual como derivable, podía destruir trabajo editorial valioso.

Si trataba todo el manual como manual, perdía la capacidad de mantener automáticamente aquello que sí conoce.

La solución empezó a aparecer al pensar el documento como un artefacto con **zonas de distinto origen y distinto ciclo de vida**.

```
User Guide
   contenido derivable
       puede regenerarse
   contenido de autor
        debe preservarse
```

Quarto encajaba bien con esta idea mediante front matter, includes y fragmentos protegidos.

IASI podía construir una base y el autor podía enriquecerla sin perder el trabajo cuando llegara una nueva materialización.

Entonces apareció un tercer estado todavía más interesante.

¿Qué ocurre si cambia algo de lo que depende un fragmento escrito por el autor?

IASI no debería reescribirlo.

Pero tampoco debería fingir que nada ha ocurrido.

Podría marcarlo como `affected` y pedir revisión.

```
dependencia cambia
      ↓
contenido derivable  → regenerar
contenido de autor   → preservar + marcar affected
```

Eso convirtió una necesidad documental en una idea mucho más general.

Los artefactos no solo tienen contenido.

Tienen **procedencia**, **dependencias** y **políticas de actualización**.

Un documento puede ser parcialmente derivable sin ser completamente regenerable.

Y esa distinción puede aparecer también en configuraciones, especificaciones, código, modelos o cualquier otro artefacto donde convivan conocimiento materializable y decisiones humanas.

Lo que quedó

El User Guide dejó de ser simplemente «documentación que podemos generar».

Pasó a ser un ejemplo de artefacto con ciclo de vida mixto.

> **Derivar lo derivable. Preservar lo autoral. Señalar lo afectado.**

La automatización deja de consistir en sobrescribir.

Empieza a consistir en saber **qué puede cambiar, qué no debe cambiar y qué necesita volver a ser revisado**.

Fuentes

- Conversaciones del 1 de septiembre de 2026 sobre User Guides y documentación derivable.
- Diseño de publicaciones Quarto con front matter e includes.
- Discusión sobre contenido `author` y estado `affected`.

title: “Reproducibilidad cuando la construcción es probabilística”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Uno de los contrastes más incómodos de IASI apareció al comparar dos cosas que la ingeniería suele querer mantener unidas:

- reproducibilidad;
- y construcción con Sistemas Inteligentes generativos.

En la mayor parte de los sistemas tradicionales esperamos que, bajo las mismas condiciones, estado y entradas, el comportamiento observable sea reproducible.

La implementación puede haber sido difícil.

Dos ingenieros pueden haber llegado a ella por caminos distintos.

Pero una vez fijado el sistema, esperamos que las mismas condiciones produzcan el mismo resultado que especificamos.

Con un Sistema Inteligente generativo aparece una segunda fuente de variabilidad.

No solo puede variar el comportamiento del sistema final si ese sistema es probabilístico.

Puede variar **el propio proceso de construcción**.

Ante la misma necesidad, dos ejecuciones pueden proponer:

```
solución A
solución B
solución C
```

con estructuras, estrategias o decisiones intermedias diferentes y, aun así, todas potencialmente válidas.

Al principio esto parecía chocar con la idea clásica de reproducibilidad.

Después recordamos que la ciencia y la ingeniería llevan mucho tiempo trabajando con procedimientos probabilísticos sin renunciar al rigor.

La aguja de Buffon ofreció un ejemplo especialmente limpio.

Podemos lanzar agujas al azar sobre un plano con líneas paralelas y utilizar la frecuencia de cruces para estimar .

Repetimos el mismo procedimiento.

Obtenemos valores distintos.

Y no por ello el experimento carece de método.

Lo importante es que existen reglas, condiciones, medidas, tolerancias y una forma de decidir si el resultado es aceptable.

La lección para IASI no era «todo puede variar».

Era casi la contraria.

> Cuando la construcción puede variar, la ingeniería debe especificar qué variabilidad admite y cómo valida sus resultados.

Eso desplaza parte del control.

En un proceso puramente determinista podemos intentar controlar exactamente el camino.

En uno probabilístico puede resultar más útil controlar:

- las entradas;
- las restricciones;
- las propiedades exigidas;
- las evidencias;
- los criterios de aceptación;
- los límites de variación;
- y la validación del resultado.

La reproducibilidad deja de significar necesariamente «obtener exactamente el mismo artefacto byte a byte».

Puede significar que distintas materializaciones satisfacen de forma verificable la misma especificación.

Lo que quedó

La ingeniería tradicional busca reproducibilidad del resultado.

La Ingeniería Aumentada debe gestionar además la variabilidad controlada de la construcción y la materialización.

```
misma necesidad
      ↓
construcciones diferentes
      ↓
criterios comunes
      ↓
resultados válidos dentro de tolerancia
```

La aleatoriedad no elimina la ingeniería.

La obliga a hacer explícito qué significa aceptar un resultado.

Fuentes

- Conversaciones del 1 de septiembre de 2026 sobre determinismo y probabilidad.
- Ejemplo pedagógico de la aguja de Buffon.
- Discusión sobre reproducibilidad del resultado frente a variabilidad de la construcción.

title: “El SI aumenta al ingeniero, no al equipo”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Hablar de «equipos aumentados» parecía suficientemente claro hasta que intentamos dibujar quién estaba aumentando realmente a quién.

La imagen fácil era algo parecido a esto:

```
equipo humano
     +
IA compartida
     ↓
equipo aumentado
```

Pero no describía bien lo que estaba ocurriendo en la práctica.

El Sistema Inteligente no se relaciona con «el equipo» como una entidad abstracta.

Se relaciona con personas concretas.

Cada ingeniero trabaja con su propio contexto, conversaciones, documentos, herramientas, hábitos, preferencias y formas de delegar.

Incluso dos personas utilizando el mismo modelo no tienen funcionalmente «la misma IA».

La relación diverge.

```
Javier + SI de Javier
Pepe   + SI de Pepe
Ana    + SI de Ana
```

El equipo está compuesto por **ingenieros aumentados**.

Eso conserva además algo importante: la coordinación organizativa sigue ocurriendo entre personas.

Javier puede decir a Pepe:

> Averigua cómo se hace esto en Maven, que yo estoy liado.

Pepe decide después cómo utilizar su SI para resolverlo.

La delegación entre compañeros no necesita convertirse en una red de agentes hablando por encima de las personas.

El SI aumenta la capacidad de Pepe para cumplir la tarea.

No sustituye la relación profesional entre Javier y Pepe.

Entonces apareció una consecuencia menos cómoda.

Si el SI se adapta a la persona, no solo puede amplificar sus capacidades.

También puede aprender, reforzar o amplificar sus defectos de trabajo.

Una persona desordenada puede producir más deprisa un desorden mayor.

Una mala costumbre puede materializarse con enorme eficiencia.

Un sesgo puede adquirir capacidad industrial.

Eso llevó a una formulación que parecía casi una advertencia de seguridad:

> **La Ingeniería Aumentada amplifica capacidades, pero también puede amplificar defectos.**

La especialización tampoco desaparece.

El hecho de que un ingeniero pueda ejecutar con ayuda tareas que antes requerían conocer una tecnología en profundidad no elimina la necesidad del experto.

Cambia su papel.

Podemos necesitar menos tiempo de ejecución manual de un especialista y más capacidad experta de validación.

Un SI puede ayudar a producir código Java.

Pero cuando importan rendimiento, concurrencia, seguridad, comportamiento de la JVM o decisiones de arquitectura, sigue haciendo falta alguien capaz de decir si aquello está bien.

El experto deja de ser únicamente quien sabe ejecutar cada paso.

Puede convertirse todavía más claramente en quien sabe **juzgar los límites de lo construido**.

Lo que quedó

La unidad organizativa no es «equipo + IA».

Es más útil pensar:

```
ingeniero + SI
      ↓
ingeniero aumentado

varios ingenieros aumentados
      ↓
equipo
```

Las responsabilidades siguen perteneciendo a las personas y a la organización.

El SI aumenta capacidad.

No absorbe automáticamente rol, autoridad ni responsabilidad.

Fuentes

- Conversaciones del 2 de septiembre de 2026 sobre equipos aumentados y especialización.
- Ejemplo de delegación entre compañeros utilizando Maven.
- Discusión sobre el cambio de papel de los expertos técnicos.

title: “El presente se convierte en el punto de partida”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

El Book I había nacido con una estructura bastante natural para un libro técnico.

Empezar por el origen.

Explicar la evolución.

Llegar poco a poco hasta el problema actual.

Era una forma razonable de construir contexto.

Pero durante los primeros días de septiembre empezamos a mirar lo que estaba ocurriendo alrededor y la estructura comenzó a parecer demasiado lenta.

Los Sistemas Inteligentes ya no se parecían al modelo mental simple con el que mucha gente había empezado a utilizarlos:

```
prompt → respuesta
```

La interacción real estaba evolucionando hacia algo bastante más parecido a:

```
intención
+ contexto
+ restricciones
+ documentos
+ herramientas
        ↓
acciones / artefactos
        ↓
validación
```

Un sistema podía leer materiales, conservar contexto, utilizar herramientas, modificar artefactos, navegar interfaces y ejecutar trabajo.

En ese escenario, dedicar muchas páginas a llegar hasta el presente corría el riesgo de ocultar precisamente la razón por la que IASI era necesario.

Entonces invertimos la pregunta editorial.

En lugar de:

> ¿Cómo contamos la historia hasta llegar aquí?

empezamos a pensar:

> ¿Qué tiene que ver el lector primero para comprender por qué existe este problema?

La respuesta fue: **el presente**.

Las capacidades actuales de OpenAI podían servir como evidencia inmediata de que la frontera ya había cambiado.

Las declaraciones y predicciones de Bill Gates podían ampliar el horizonte y mostrar que el cambio no se estaba percibiendo como una pequeña mejora de herramientas, sino como una transformación mucho más amplia.

Pero con una precaución importante.

No queríamos utilizar a Gates como autoridad que demostrara nuestra tesis.

Su función era otra.

Ser un síntoma contemporáneo de la velocidad y profundidad con que el cambio estaba siendo percibido.

A partir de ahí apareció una estructura de preámbulo en tres movimientos:

```
1. OpenAI
   qué está ocurriendo ya

2. Bill Gates
   hasta dónde parece proyectarse el cambio

3. Lo nuestro
   qué preguntas de ingeniería aparecen
```

Y en el tercer movimiento no debíamos vender todavía la solución.

Antes había que crear la necesidad de formular bien el problema.

La pregunta central se volvió mucho más nítida:

> **¿Cómo hacemos ingeniería cuando los Sistemas Inteligentes participan en la propia ingeniería?**

No «cuando ayudan a programar».

No «cuando generan código».

Cuando participan en la propia ingeniería.

Si leen decisiones, interpretan restricciones, utilizan herramientas, producen artefactos y modifican el sistema, ya no están fuera del proceso.

Son parte del proceso.

Y esa participación debe ser diseñada, gobernada, trazada y validada.

Lo que quedó

El Book I dejó de necesitar un largo camino para justificar el presente.

El presente podía convertirse en la puerta de entrada.

```
presente observable
      ↓
preguntas incómodas
      ↓
necesidad de ingeniería
      ↓
IASI
```

La introducción no debe empezar demostrando que IASI tiene razón.

Debe conseguir algo más importante:

que la pregunta resulte inevitable.

Fuentes

- Conversaciones del 5 de septiembre de 2026 sobre la reestructuración del Book I.
- Evolución observada de ChatGPT/OpenAI hacia documentos, contexto, herramientas, acciones y artefactos.
- Discusión sobre el uso de declaraciones de Bill Gates como síntoma y no como autoridad.

title: “Los roles son necesarios; los puestos y los recursos son contingentes”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Mientras intentábamos describir la organización de la Ingeniería Aumentada apareció una confusión muy fácil de cometer.

Hablar de roles llevaba casi automáticamente a imaginar personas distintas.

Un rol funcional.

Otro metodológico.

Otro técnico.

Y, sin darnos cuenta, el dibujo empezaba a parecer un organigrama.

Pero un rol no es una persona.

Tampoco es un puesto.

Ni implica necesariamente dedicar un recurso completo.

La distinción se volvió mucho más fácil de explicar al recordar los Six Thinking Hats de Edward de Bono.

No porque los sombreros de De Bono correspondan a los roles de IASI.

No existe esa equivalencia.

La idea útil era más sencilla: **una misma persona puede ponerse distintos sombreros**.

Cuando actúa desde un rol, debe saber desde qué rol está decidiendo.

```
misma persona
   sombrero / rol A
   sombrero / rol B
   sombrero / rol C
```

Y hay otro detalle importante.

No llevamos dos sombreros a la vez.

Una persona puede desempeñar varios roles, pero ante una decisión concreta conviene saber cuál está ejerciendo.

Eso ayuda a separar preguntas que de otro modo se mezclan:

- ¿qué necesita el producto?;
- ¿qué exige la metodología?;
- ¿qué permite técnicamente el sistema?;
- ¿quién tiene autoridad para aceptar una excepción?;
- ¿quién valida el resultado desde cada perspectiva?

En una organización grande esos roles pueden terminar asignados a personas o equipos diferentes.

En un proyecto pequeño una misma persona puede ejercer varios.

En algunos momentos uno de esos roles puede apoyarse intensamente en un Sistema Inteligente.

Nada de eso cambia la necesidad de que la **función exista**.

La estructura organizativa debe distinguir lo necesario de la forma concreta en que se materializa.

Y de ahí salió una frase que terminó condensando toda la discusión:

> **Los roles son necesarios; los puestos y los recursos son contingentes.**

La metodología necesita determinadas responsabilidades.

No necesita imponer un organigrama universal.

Lo que quedó

IASI puede especificar roles sin prescribir cuántas personas debe tener una organización.

```
rol necesario
     ↓
puede materializarse como
     ↓
persona / varias personas / parte de un puesto / equipo
```

Esto resulta especialmente importante en Ingeniería Aumentada, porque la capacidad disponible puede cambiar mucho según las personas y los Sistemas Inteligentes que las aumentan.

La metodología debe fijar **qué funciones no pueden desaparecer**.

La organización decidirá después cómo las cubre.

Fuentes

- Conversaciones del 5 de septiembre de 2026 sobre estructuras organizativas de IASI.
- Six Thinking Hats de Edward de Bono utilizado únicamente como recurso pedagógico para distinguir persona, puesto y rol.
- Formulación consolidada durante la discusión: «Los roles son necesarios; los puestos y los recursos son contingentes.»

title: “El punto de inflexión depende de la escala”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

A veces mirar IASI desde demasiado lejos produce una sensación extraña.

Queremos construir una metodología completa para Ingeniería Aumentada.

Comparado con ese objetivo, cualquier avance concreto parece pequeño.

Un modelo de inputs.

Una política de artefactos.

Una estructura organizativa.

Un mecanismo de publicación.

Una decisión sobre roles.

Todo puede parecer apenas el comienzo de una curva todavía pegada al suelo.

La función sigmoidal apareció inicialmente como una forma intuitiva de pensar ese progreso.

```
inicio lento
   ↓
aceleración
   ↓
punto de inflexión
   ↓
maduración
```

Pero entonces hicimos zoom sobre la parte aparentemente plana.

Y allí también había cambios importantes.

Dentro del desarrollo del modelo conceptual podía aparecer otra dinámica de aceleración.

Dentro de una estructura organizativa, otra.

Dentro de una única decisión de ingeniería, otra.

Eso llevó a una formulación más cuidadosa que decir simplemente «la sigmoide es fractal», porque matemáticamente esa afirmación sería demasiado fuerte.

La idea útil era otra:

> **El proceso observado puede presentar autosimilitud sigmoidal a distintas escalas.**

```
IASI completo
  modelo conceptual
      estructura organizativa
          decisión concreta
```

En cada escala podemos encontrar una fase inicial, una aceleración, un cambio de régimen y una estabilización relativa.

Por eso dos afirmaciones aparentemente contradictorias pueden ser simultáneamente ciertas:

> Hemos avanzado muy poco.

Y:

> Hemos avanzado una barbaridad.

Desde la escala de la metodología completa seguimos al principio.

Desde la escala de algunos problemas concretos ya hemos atravesado varios puntos de inflexión.

Esto también introdujo una advertencia epistemológica.

Observar únicamente un tramo local puede engañarnos sobre la dinámica global.

Una zona casi horizontal puede ser el comienzo de una curva mayor.

Una aceleración espectacular local puede seguir siendo pequeña dentro del sistema completo.

El punto de inflexión, por tanto, no es una propiedad narrativa absoluta del proyecto.

Depende de la escala desde la que lo observamos.

Lo que quedó

La sigmoidal no se convirtió en una ley de IASI.

Se convirtió en un instrumento conceptual para pensar cambio, madurez y perspectiva.

Nos permite decir con más precisión algo que el trabajo diario llevaba tiempo mostrando:

```
avance local enorme

metodología global terminada
```

Y también lo contrario:

```
metodología global todavía inicial

no hemos progresado
```

Cambiar la escala cambia lo que vemos.

No necesariamente lo que ha ocurrido.

Fuentes

- Conversación del 6 de septiembre de 2026 sobre la función sigmoidal y el progreso de IASI.
- Discusión sobre autosimilitud por escala y dependencia del punto de inflexión respecto del nivel de observación.

title: “Astra se convierte en un caso real para IASI”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

La idea inicial era sencilla.

Solicitar acceso a Project Astra como Trusted Tester.

Astra resultaba especialmente interesante por las capacidades hacia las que apunta: multimodalidad, memoria, herramientas, interacción continua y actuación sobre el entorno.

Era, en muchos sentidos, una buena ventana a la siguiente generación de Sistemas Inteligentes.

La primera tentación fue tratar la candidatura como cualquier solicitud de acceso a un programa experimental.

Explicar experiencia.

Explicar interés.

Explicar posibles casos de uso.

Pero cuanto más intentábamos describir por qué Astra era relevante para nosotros, más evidente se volvía que la relación podía formularse de otra manera.

Google DeepMind está trabajando en la **tecnología** que permite a un Sistema Inteligente observar, recordar, utilizar herramientas e intervenir sobre un entorno.

IASI está investigando qué ocurre con la **ingeniería** cuando Sistemas Inteligentes con esas capacidades participan activamente en ella.

No eran trabajos equivalentes.

Tampoco competidores.

Eran capas distintas de un problema que empezaban a tocarse.

```
Project Astra
capacidades del SI
        ↓
participación real en tareas
        ↓
IASI
cómo ingenierizar esa participación
```

Eso cambió el papel de Astra dentro de la candidatura.

Dejó de ser simplemente una tecnología interesante que queríamos probar.

Podía convertirse en un **caso real de evolución de IASI**.

No un escenario sintético diseñado para confirmar nuestras ideas.

Un Sistema Inteligente real, cambiante, con capacidades nuevas, utilizado dentro de trabajo real de ingeniería.

La relación empezó a verse de forma bidireccional.

Astra podía ayudar a IASI exponiéndolo a capacidades y problemas que todavía no habíamos experimentado.

IASI podía ofrecer una perspectiva metodológica sobre cuestiones que inevitablemente aparecen cuando esas capacidades entran en un proceso de ingeniería:

- confianza;
- delegación;
- autoridad;
- autonomía;
- responsabilidad;
- sesgos;
- trazabilidad;
- validación;
- límites de actuación.

Aquí apareció también una formulación importante.

Cuando el Sistema Inteligente participa activamente en la ingeniería, no «puede convertirse» en parte del proceso.

Es parte del proceso.

Y esa participación debe ser engineered and managed.

Eso no significa borrar la diferencia entre humano y SI.

Al contrario.

Podemos pensar en humano y SI operando juntos como una **augmented engineering unit**, manteniendo explícitamente distintas responsabilidades, autoridad y capacidad de respuesta.

Lo que quedó

La candidatura a Astra dejó de ser únicamente una petición de acceso.

Se convirtió en una prueba de una idea central de IASI:

> Una metodología para Ingeniería Aumentada necesita enfrentarse a Sistemas Inteligentes reales, no solo describirlos desde fuera.

Si Astra evoluciona, IASI tendrá que aprender de esa evolución.

Y si IASI sirve para algo, debería ayudarnos a formular mejor qué significa incorporar esas capacidades a la ingeniería sin perder control, responsabilidad ni trazabilidad.

Fuentes

- Conversaciones del 7 de septiembre de 2026 sobre la candidatura a Project Astra Trusted Tester.
- Iteraciones V1 y V2 de la candidatura.
- Definición de IASI como iniciativa independiente que investiga, define y crea un nuevo enfoque de ingeniería para entornos donde los Sistemas Inteligentes participan activamente.

title: “La estructura del repositorio también es arquitectura”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Queríamos hacer algo aparentemente pequeño con `iasi-quarto`.

Ordenar el repositorio.

Había documentación, software, outputs, configuración y material de publicación. Parte de la estructura procedía de etapas anteriores y empezaba a no representar bien lo que el proyecto era.

La primera corrección parecía casi cosmética:

```
src/ → software/
```

Pero no era solo un cambio de nombre.

`src` sugería que aquello era «el código fuente» del repositorio.

Sin embargo, el repositorio contenía más de un tipo de producto y podía contener más de un proyecto software.

`software/` describía mejor la categoría.

Así apareció una estructura deliberadamente redundante:

```
software/
  iasi-quarto/
```

Dentro del repositorio `iasi-quarto`.

La duplicación del nombre dejó de molestar cuando distinguimos los niveles:

```
repo / contexto     = iasi-quarto
categoría           = software
proyecto software   = iasi-quarto
```

Algo parecido ocurrió con la documentación.

`docs/01-user-guide/` heredaba una numeración que tenía sentido cuando pensábamos en una secuencia.

Pero aquello no era un capítulo.

Era un proyecto.

Por eso pasó conceptualmente a:

```
docs/user-guide/
```

Y entonces apareció una pregunta mucho más importante.

Si `docs/user-guide/` es un proyecto IASI, ¿qué declara que lo es?

La respuesta ya existía:

```
_iasi.yml
```

No `.iasi/`.

`.iasi/` puede contener estado de trabajo, caché, índices, material temporal o información regenerable.

`_iasi.yml` es otra cosa.

Es la **declaración**.

El proyecto existe como proyecto IASI porque se declara.

Y al mirar hacia arriba descubrimos el hueco.

El propio root del repositorio también necesitaba una declaración.

No porque fuera un «multiproject».

Podía tener uno, varios o ningún hijo de cierto tipo en un momento dado.

La cardinalidad no debía definir su identidad.

Necesitábamos el tipo base:

```yaml
project:
  type: project
```

La idea parecía pequeña pero tenía una consecuencia grande.

`book`, `guide`, `r-package` u otros tipos pueden ser especializaciones de proyecto.

Todo book es un project.

No todo project es un book.

La estructura física empezó a expresar esa jerarquía conceptual.

Después llegó `release` y terminó de separar niveles que todavía estaban mezclados.

El proyecto hijo sabe construir y publicar sus outputs.

Pero no necesita saber que vive dentro de un repositorio Git.

`release`, en cambio, pertenece al ámbito del repositorio porque reúne aquello que el repositorio decide liberar.

```
repo
  docs/
     user-guide/      ← proyecto
  software/
     iasi-quarto/     ← proyecto
  release/             ← ensamblado del repo
```

Por eso una operación de release debe localizar el Git root, combinar la configuración raíz con la del proyecto local y reunir material ya construido o publicado.

No reconstruirlo.

Promoverlo.

Y `release/` tampoco tenía por qué reproducir las categorías internas del source tree.

La estructura liberada debía estar orientada a productos:

```
release/
  index.html
  user-guide/
  iasi-quarto/
```

La publicación en GitHub Pages confirmó otra consecuencia interesante.

Un único repositorio podía exponer varios productos web en subrutas y, al mismo tiempo, distribuir archivos no web desde el mismo árbol de release.

Lo que quedó

Empezamos moviendo carpetas.

Terminamos distinguiendo cuatro conceptos:

```
proyecto
repositorio
producto
release
```

No son sinónimos.

Y cuando los mezclamos, la estructura del disco empieza a mentir sobre la arquitectura.

> **La estructura del repositorio también comunica el modelo del sistema.**

Ordenarla no era una operación estética.

Era hacer explícita una arquitectura que ya estaba apareciendo.

Fuentes

- Conversaciones del 9 de septiembre de 2026 durante la reestructuración de `iasi-quarto`.
- Estructura publicada del repositorio `iasi-quarto`.
- Discusión sobre `_iasi.yml`, `.iasi/`, Git root y responsabilidad de `release`.

title: “Congelar la web no significa terminar IASI”

Este capítulo forma parte del recorrido de IASI. La narración completa continúa en el documento siguiente.

Llevábamos varios días intentando «congelar» distintas partes de IASI.

La palabra tenía un peligro.

Podía interpretarse como terminar.

Si congelar la web significaba completar todos los manuales, productos, subproductos, páginas conceptuales y documentación pendiente, entonces la web no se congelaría hasta que IASI estuviera prácticamente terminado.

Una condición bastante absurda para una iniciativa que precisamente está evolucionando.

El cambio apareció al formular mejor qué queríamos congelar.

No el contenido final.

La representación pública del sistema.

La web ya tenía dos destinos preparados para reconocer explícitamente que algo todavía no está hecho:

```
status/to-be-done
status/under-construction
```

Eso cambiaba completamente el criterio.

Una página podía estar vacía o en construcción y, aun así, ocupar correctamente su lugar en el mapa.

Un menú podía mostrar una capacidad prevista aunque su desarrollo todavía no hubiera empezado.

Lo importante era que la web cumpliera otra función:

> Dar una imagen global, coherente y suficientemente completa de qué es IASI, qué contiene y hacia dónde se está construyendo.

Entonces el trabajo dejó de parecer «rellenar páginas».

Se convirtió en arquitectura de información.

Empezamos a revisar el menú principal.

Algunas etiquetas ya no representaban bien lo que contenían.

`Artefactos`, por ejemplo, agrupaba `IASI Tools Dev` e `IASI RStudio`.

Pero esas cosas podían entenderse mejor como **Productos**.

La definición empezó a afinarse:

```
Subproductos
= cosas que IASI necesitó y creó durante su construcción

Productos
= cosas creadas inicialmente para IASI que,
  con ajustes, pueden tener vida propia fuera de IASI
```

La frontera no dependía por tanto de quién fuera el usuario original.

Ambos podían nacer de una necesidad interna.

La diferencia era su capacidad de **emanciparse**.

Un filtro específico, una utilidad interna o un servidor MCP podían aparecer como subproductos del trabajo.

IASI Quarto, IASI Lua, IASI Tools Dev o IASI RStudio podían presentarse como productos cuando adquirían suficiente autonomía para ser útiles fuera del propio proyecto.

El orden del menú también empezó a importar.

Productos antes que Subproductos resultaba más natural para alguien que llega desde fuera.

Primero ve aquello que puede reconocer como una unidad utilizable.

Después puede descubrir las piezas que fueron apareciendo durante la construcción.

Otro problema apareció con `Metodología`.

El menú solo contenía:

```
Agentics
Adapters
```

La etiqueta hacía una afirmación involuntaria: parecía decir que la metodología IASI **era** Agentics + Adapters.

Y eso ya sabíamos que era falso.

La metodología es mucho más amplia.

Agentics y Adapters son enfoques, mecanismos o piezas dentro de ella.

Cambiar el nombre del menú a `Enfoques` resolvía el problema sin inventar todavía una taxonomía definitiva.

```
Enfoques
  Agentics
  Adapters
```

También vimos que `Intro`, `Sobre IASI`, `Principios`, `Manifiesto` y `The Challenge` estaban repartiendo por el navbar algo que conceptualmente pertenecía a una misma zona de presentación de IASI.

La revisión del menú dejó de ser decoración.

Era una prueba de coherencia conceptual.

Si no sabíamos dónde colocar una cosa, probablemente todavía no habíamos decidido bien qué era.

Congelar sin fingir

El criterio final empezó a parecer mucho más sano:

La web puede contener andamios.

Lo que no puede contener son **agujeros conceptuales involuntarios**.

Puede decir:

```
Under construction
```

Lo que no debe hacer es ocultar una parte importante de IASI simplemente porque todavía no está terminada.

Puede mostrar una ruta hacia algo pendiente.

Lo que no debe hacer es enviar al usuario a un enlace roto.

Puede tener profundidad desigual.

Lo que debe ofrecer es una imagen global suficientemente completa.

```
estructura estable
+ navegación válida
+ mapa completo
+ estado explícito
= web congelable
```

Lo que quedó

Congelar la web no significa congelar el desarrollo de IASI.

Tampoco significa declarar terminado lo que sigue en construcción.

Significa algo mucho más concreto:

> **Congelamos la representación del sistema, no el estado de desarrollo del sistema.**

A partir de ese momento, `to-be-done` y `under-construction` dejaron de parecer defectos temporales de la web.

Se convirtieron en parte legítima de una representación honesta del estado de IASI.

Y la navegación empezó a cumplir una segunda función.

No solo permite llegar a páginas.

Cuenta qué creemos que IASI es.

Fuentes

- Conversaciones del 9 de septiembre de 2026 durante la congelación de `iasi-home`.
- Páginas existentes `status/to-be-done` y `status/under-construction`.
- Revisión del navbar de `iasi-home` y discusión sobre Productos, Subproductos, Enfoques y estructura pública del proyecto.